



|        |          |
|--------|----------|
| 项目批准号  | 62202339 |
| 申请代码   | F0206    |
| 归口管理部门 |          |
| 收件日期   |          |



2025 11622 02 33 9

# 国家自然科学基金 资助项目结题/成果报告

资助类别：青年科学基金项目（C类）[原青年科学基金项目]

亚类说明：

附注说明：

项目名称：抗强攻击者的多方神经网络隐私计算

负责人：冯琦 BRID：08801.00.72355

电子邮件：fengqi.whu@whu.edu.cn 电话：13007192940

依托单位：武汉大学

联系人：熊鸣 电话：027-68772100

资助经费：30.0000（万元） 执行年限：2023.01-2025.12

填表日期：2026年01月02日

国家自然科学基金委员会制（2025年）



## 项目摘要

### 中文摘要:

数据隐私是神经网络技术健康发展的重要保障,隐私计算可在保护数据机密性的前提下,提供敏感数据运算功能,有效平衡数据隐私性和可用性之间的矛盾。但目前神经网络隐私计算存在安全假设较强、扩展性不足、难以支持复杂训练模型等问题。本项目拟利用零知识混淆电路、承诺不经意传输、带验证秘密共享、同态加密等密码技术,研究抗强攻击者的多方神经网络隐私计算。主要研究内容包括:1) 研究大整数域和比特域安全多方计算基础部件,设计安全耦合的运算域转换模块;2) 构造支持不同神经网络预测和训练模型的多方隐私计算方案,满足恶意敌手模型下的可证明安全性;3) 实现基于标准数据集的多方神经网络隐私计算实例化方案,开展正确性和性能测试。本项目兼顾安全、隐私、效率等实际应用需求,力图打破数据壁垒,发展数据流动中的隐私保护理论,健全神经网络数据安全防护体系,为大规模数据应用提供理论基础和技术支撑。

### Abstract:

Data privacy is the cornerstone for the healthy development of neural networks. The privacy-preserving calculation can protect the confidentiality of data while providing the functionality of computing over sensitive data, balancing the contraction between data privacy and data usability. Drawbacks in existing solutions include strong security assumptions, low extensibility, and being challenging to support complicated training algorithms. Therefore, this project focuses on the multi-party privacy-preserving neural network against strong adversaries, potentially using cryptographic technologies involving zero-knowledge garbled circuit, committed oblivious transfer, authenticated secret sharing, homomorphic encryption, and so on forth. Specifically, 1) we will first study secure multi-party calculation toolkits over arithmetic and bool fields, and secure conversion models between different fields; 2) We will construct multi-party privacy-preserving computing schemes for prediction and training algorithms of different neural networks, and formally prove the security against the malicious adversary; 3) We will implement the privacy-preserving neural network schemes based on standard test datasets, and consequently test the correctness and performance. Taking into consideration practical requirements, e.g., security, privacy, efficiency, etc., this project aims to break the data island and develop the privacy protection theory in the data flow. The results of this project will lay a critical theoretical foundation on the data privacy of neural networks and provide efficient technical support for the healthy development of large-scale data applications.

**关键词 (用分号分开):** 大数据隐私保护; 协同计算; 信息论安全; 隐私模型训练; 人工智能安全



**Keywords (separated by;):** Privacy Preservation; Collaborative Computation; Information Theoretically Security; Privacy-Preserving Training; Artificial Intelligence Security

NSFC-REPORT-2025



## 结题摘要

**中文摘要（对项目的背景、主要研究内容、重要结果、关键数据及其科学意义等做简单概述）：**

本项目围绕“多方计算基础部件的跨运算域有机融合”和“隐私计算执行与验证的分离优化模式”两大关键科学问题四个关键科学问题，从“抗恶意敌手攻击的多方神经网络隐私计算基础部件”、“抗恶意敌手攻击的多方神经网络隐私计算安全协议”和“基于标准数据集的多方神经网络隐私计算系统仿真测试”三个方面开展研究，取得了一批支持不同神经网络模型的高效多方隐私计算研究成果，包括计算过程可验证、数据匿名获取、大规模数据融合计算，强化了神经网络模型数据隐私保护能力，保障数据安全流动。立项后，在项目相关领域重要期刊和会议上发表高质量论文10篇，其中，其中，IEEE Trans和CCF推荐期刊/会议9篇；共申请国家发明专利2件，其中授权1件，研究团队共协助培养博士研究生6名、硕士研究生9名。本项目在抗强攻击者的多方神经网络隐私计算及系统设计方面取得了突破，进而推动了国家在数据安全方面的创新发展，保障国家人工智能领域产业发展的安全可控。项目研究工作按计划进行，完成了项目研究任务，实现了项目预期目标。

**Abstract (Brief description of research background, main methods, contributions, and research data):**

This project addresses two core scientific problems: the organic integration of multi-party computation (MPC) building blocks across heterogeneous computation domains and the decoupled optimization paradigm for privacy-preserving computation execution and verification, involving four key scientific questions in total. The research is carried out along three main directions: (1) privacy-preserving MPC building blocks for multi-party neural networks against malicious adversaries, (2) secure MPC protocols for multi-party neural network computation under malicious security, and (3) system-level simulation and evaluation of privacy-preserving multi-party neural network computation based on standard benchmark datasets. The project has produced a series of efficient MPC techniques supporting diverse neural network models, including verifiable computation, privacy-preserving data acquisition, and large-scale data fusion computation. These results significantly enhance the data privacy protection capability of neural network models and ensure secure and controlled data circulation. Since the project initiation, 10 high-quality papers have been published in leading journals and conferences in related areas, including 9 papers in IEEE Transactions journals and CCF-recommended journals or conferences. In addition, 2 national invention patents have been filed, one of which has been granted. The research team has also contributed to the training of 6 Ph.D. students and 9 master's students. The project has achieved key breakthroughs in privacy-preserving multi-party neural network computation and system design against strong adversaries, promoting national innovation in data security and supporting the secure and controllable development of artificial intelligence industries. The research has progressed as planned, successfully completed all tasks, and achieved the expected objectives.

**关键词（用分号分开）：** 大数据隐私保护； 协同计算； 信息论安全； 隐私模型训练； 人工智能安全



**Keywords (separated by;):** Privacy Preservation; Collaborative Computation; Information Theoretically Security; Privacy-Preserving Training; Artificial Intelligence Security

NSFC-REPORT-2025



## 正文

《结题/成果报告》正文分为两个部分：**结题部分**和**成果部分**。请按照《结题/成果报告》填报说明及撰写要求填写。

### （一）结题部分

#### 1. 研究计划执行情况概述。

##### 1.1 按计划执行情况。

本项目从神经网络隐私计算方案中存在的难以抵抗恶意敌手攻击、可扩展性和灵活性不足、不支持复杂计算模式等问题出发，按照“基础部件→安全协议→评估测试”的研究思路，从“抗恶意敌手攻击的多方神经网络隐私计算基础部件”、“抗恶意敌手攻击的多方神经网络隐私计算安全协议”和“基于标准数据集的多方神经网络隐私计算系统仿真测试”三个方面开展研究，构建抗强攻击者的多方神经网络隐私计算。本项目从 2023 年 1 月开始实施，为期 3 年。目前项目的研究工作已经按照计划圆满完成，并取得丰硕研究成果。

2023 年度，项目组在抗恶意敌手攻击的多方神经网络隐私计算基础部件方面，设计一种基于多验证者零知识证明（MVZK）的无状态确定性随机数生成协议，解决隐私神经网络建模中随机数生成的正确性、一致性问题，显著降低通信复杂度，使协议在分布式场景中具备高效恶意行为验证能力；在多方神经网络隐私计算安全协议方面，面向基于区域的卷积神经网络模型，设计了一种三方快速的隐私保护计算方案，结合随机数掩码盲化和安全乘-加转化结构，确保原始图像数据的隐私性；在多方神经网络隐私计算系统仿真测试方面，针对仿真方案在部署时所需的密钥管理需求，提出了一种高效的 EdDSA 签名方案，恶意行为可识别中止机制，解决了多方协作中的身份验证问题。

2024 年度，项目组在抗恶意敌手攻击的多方神经网络隐私计算基础部件方面，围绕训练数据安全获取开展工作，并取得重要成果，项目组利用非对称标量积保持加密技术，设计了一种面向单使用者的可验证动态密文检索方案 VFMKQ，在保障加密索引和检索陷门安全性、隐私性、可验证性的同时，结合正排索引与倒排索引结构，提高了方案的检索效率；在多方神经网络隐私计算安全协议方面，项目组围绕抗恶意敌手行为和大规模复杂模型支持两条研究思路展



开研究。首先，项目组对现有神经网络隐私保护方案进行了深入调研与对比，通过融合“特权方+2 个辅助方”结构，项目组设计了具有鲁棒性的安全比较、安全除法和安全乘法协议，保持一个辅助方掉线仍能完成计算，有效缓解恶意行为带来的计算错误；同时，面向生成式预训练模型（GPT）模型，项目组设计一种针对 Transformer 的隐私保护安全多方推理框架，探索多方神经网络隐私计算安全协议在大规模神经网络模型下的可行性和可推广性。

2025 年度，项目组在多方神经网络隐私计算安全协议方面提出一系列创新方案，面向长短时记忆神经网络 LSTM 模型，项目组融合“特权方+2 个辅助方”结构，设计了一种高效的序列到序列转换方案，结合函数秘密分享和傅里叶级数，有效防止自然语言处理中数据泄露问题；项目组在多方神经网络隐私计算系统仿真测试方面，将分布式密钥签名方案优化为更通用的 ECDSA 适配方案，创新提出一种高效的三方 ECDSA 签名方案，提高在分布式网络中的兼容性和高效性，同时，项目组集成了神经网络模型前端/后端适配层的相关模块，构建了标准化的隐私计算协议接口及其管理组件，为了降低隐私神经网络的使用门槛，项目组基于 TensorFlow 框架进行设计，借助 TensorFlow 的高可扩展性、前端 Python 计算定义层、后端 C++ 计算执行层、高性能图执行引擎、优化遍 Pass 可定制等能力，从而设计隐私神经网络的分布式计算框架，并基于该框架对理论方案进行正确性和性能测试。

总体而言，研究计划按照预定的计划执行情况有序推进，取得了显著的研究成果，为基于区块链的数据安全共享在安全性和功能性方面提供了重要技术支持。

## 1.2 研究目标完成情况。

本项目围绕“多方计算基础部件的跨运算域有机融合”和“隐私计算执行与验证的分离优化模式”两大关键科学问题四个关键科学问题，从“抗恶意敌手攻击的多方神经网络隐私计算基础部件”、“抗恶意敌手攻击的多方神经网络隐私计算安全协议”和“基于标准数据集的多方神经网络隐私计算系统仿真测试”三个方面开展研究，取得了一批支持不同神经网络模型的高效多方隐私计算研究成果，包括计算过程可验证、数据匿名获取、大规模数据融合计算，强化了神经网络模型数据隐私保护能力，保障数据安全流动。立项后，在项目相关领域重要期刊和会议上发表高质量论文 10 篇，其中，其中，IEEE Trans 和 CCF 推荐期刊/会议 9 篇；



共申请国家发明专利2件,均已获得授权,研究团队共协助培养博士研究生6名、硕士研究生9名。本项目在抗强攻击者的多方神经网络隐私计算及系统设计方面取得了突破,进而推动了国家在数据安全方面的创新发展,保障国家人工智能领域产业发展的安全可控。项目研究工作按计划进行,完成了项目研究任务,实现了项目预期目标。

## 2. 研究工作主要进展、结果和影响。

### 2.1 主要研究内容。

针对面向神经网络模型中数据隐私保护需求,以保障数据隐私性、抵抗恶意敌手攻击、实现系统能效平衡为导向,本项目研究了研究抗恶意敌手攻击的多方神经网络隐私计算基础部件、多方神经网络隐私计算安全协议、多方神经网络隐私计算系统仿真测试。

1. 在抗恶意敌手攻击的多方神经网络隐私计算基础部件方面,项目组设计一种基于多验证者零知识证明(MVZK)的无状态确定性随机数生成协议,解决隐私神经网络建模中随机数生成的正确性、一致性问题,显著降低通信复杂度,使协议在分布式场景中具备高效恶意行为验证能力。针对训练数据获取时查询模式的隐私保护需求,项目组利用非对称标量积保持加密技术,设计了一种面向单使用者的可验证动态密文检索方案 VFMKQ,在保障加密索引和检索陷门安全性、隐私性、可验证性的同时,结合正排索引与倒排索引结构,提高了方案的检索效率;通过融合“特权方+2个辅助方”结构,项目组设计了具有鲁棒性的安全比较、安全除法和安全乘法协议,保持一个辅助方掉线仍能完成计算,有效缓解恶意行为带来的计算错误。

2. 在多方神经网络隐私计算安全协议方面,面向基于区域的卷积神经网络模型,项目组设计了一种三方快速的隐私保护计算方案,结合随机数掩码盲化和安全乘-加转化结构,确保原始图像数据的隐私性;面向长短时记忆神经网络 LSTM 模型,项目组融合“算术电路+布尔电路+函数秘密分享”混合框架,设计具有隐私保护的序列到序列转换方案,结合分布式比较函数和傅里叶级数,有效防止自然语言处理中数据泄露问题;面向生成式预训练模型(GPT)模型,项目组设计一种针对 Transformer 的隐私保护安全多方推理框架,探索多方神经网络隐私计算安全协议在大规模神经网络模型下的可行性和可推广性。



3. 在多方神经网络隐私计算系统仿真测试方面，针对仿真方案在部署时所需的密钥管理和身份确认需求，项目组提出了一种高效的三方（Elliptic Curve Digital Signature Algorithm, ECDSA）签名方案，基于复制秘密共享和可识别中止机制，解决了多方协作中的身份验证与数据访问授权问题。项目组集成了神经网络模型前端/后端适配层的相关模块，构建了标准化的隐私计算协议接口及其管理组件。同时为了降低隐私神经网络的门槛，项目组基于 TensorFlow 框架进行设计，借助 TensorFlow 的高可扩展性、前端 Python 计算定义层、后端 C++ 计算执行层、高性能图执行引擎、优化遍 Pass 可定制等能力，从而设计隐私神经网络的分布式计算框架，并基于该框架对理论方案进行正确性和性能测试。

## 2.2 取得的主要研究进展、重要结果、关键数据等及其科学意义或应用前景。

### 2.2.1 抗恶意敌手攻击的多方神经网络隐私计算基础部件

项目组围绕神经网络模型的隐私计算需求，设计面向多验证者的零知识证明协议，确保在一般恶意敌手大多数场景下计算结果的可验证；围绕神经网络模型隐私计算的数据安全获取需求，设计可验证且前向安全的多关键词隐私数据查询，为多方神经网络隐私计算协议提供高质量可验证的敏感数据源；围绕神经网络模型算法中的乘法、除法、比较等运算，设计一种具有特权方的恶意敌手模型多方隐私保护计算协议，兼顾隐私性、高效性和恶意敌手鲁棒性。

#### （1）面向多验证者的零知识证明协议

安全模型。项目组在通用可组合性（UC）框架下证明了协议的安全性，假设存在一个静态的、恶意的敌手，其可以腐化 $n$ 个参与方中的最多 $n - 1$ 个。如果对于任意的概率多项式时间（PPT）敌手  $\mathcal{A}$ ，都存在一个 PPT 敌手  $\mathcal{S}$ （称为模拟器），使得对于任意带有辅助输入 $z$ 的 PPT 环境  $\mathcal{Z}$ ，在真实世界执行（即各方与  $\mathcal{A}$  交互并执行  $\Pi$ ）和理想世界执行（即各方与  $\mathcal{S}$  和  $\mathcal{F}$  交互）中， $\mathcal{Z}$  的输出分布在计算上是不可区分的，则称协议  $\Pi$  UC-实现了理想功能  $\mathcal{F}$ 。环境  $\mathcal{Z}$  是一个强大的实体，它对敌手  $\mathcal{A}$  拥有完全控制权，并且可以选择所有参与方的输入并查看它们的输出。

项目组用  $P_1, \dots, P_n$  表示  $n$  个参与方，并用  $J$  表示被腐化的参与方集合。本方案考虑中止安全性，即腐化的参与方可以获得输出，而诚实的参与方可能无法



获得输出。在这种情况下，理想世界中的对手首先接收输出，然后向理想功能发送  $(\text{continue}, i)$  或  $(\text{abort}, i)$ ，其中  $i \notin \mathcal{I}$ ，用于指示理想功能要么将输出传递给  $P_i$ ，要么向  $P_i$  发送  $\text{abort}$ 。

首先定义多验证者信息论安全消息认证码的概念，它是基于 VOLE 的单验证者零知识协议的推广。需要认证的值位于  $\mathbb{F}_2$  或  $\mathbb{F}_{2^\kappa}$  中，认证在二元扩展域  $\mathbb{F}_{2^\kappa}$  上进行。具体而言，设  $\mathcal{V}_1, \dots, \mathcal{V}_N$  表示  $N$  个验证者，且设  $\Delta i \in \mathbb{F}_{2^\kappa}$  为每个验证者  $\mathcal{V}_i$  独有的均匀全局密钥。一个由证明者  $\mathcal{P}$  知道的值  $x \in \mathbb{F}_2$  或  $\mathbb{F}_{2^\kappa}$  被认证为  $[[x]]_2 = (x, m_1, \dots, m_N), k_1, \dots, k_N$ ，其中每个  $m_i = k_i + x \cdot \Delta i \in \mathbb{F}_{2^\kappa}$ ，并且具有相同的  $x$  值。每个验证者  $\mathcal{V}_i$  持有一个本地 MAC 密钥  $k_i, i \in [N]$ ，而证明者  $\mathcal{P}$  保留该秘密值以及相应的 MAC 标签  $(x, m_1, \dots, m_N)$ 。

认证值均支持加法同态性。例如，给定在  $\mathbb{F}_{2^\kappa}$  上认证的比特，即  $[[x_1]]_2, \dots, [[x_\ell]]_2$ ，以及公共系数  $c_1, \dots, c_\ell, c \in \mathbb{F}_{2^\kappa}$ ，各方可以本地计算  $[[y]]_2 = \sum_{i=1}^{\ell} c_i \cdot [[x_i]]_2 + c$ 。下面，定义四个将在本方案中使用的宏。

- **Random:** 为了生成一个认证值  $[[r]]_2$ ，其中  $r \in \mathbb{F}_2$  是一个均匀随机值， $\mathcal{P}$  和各验证者  $\mathcal{V}$  可以以无状态且确定性的方式调用  $PCF.Evalmv$ ，使用预计算的  $k_0$  作为  $\mathcal{P}$  的输入，以及  $k_i$  作为每个验证者  $\mathcal{V}_i$  的输入，其中  $i \in [N]$ 。我们使用 **Random** 来表示此宏。
- **Assign:** 给定来自  $\mathcal{P}$  的输入  $x \in \mathbb{F}_2$ ， $\mathcal{P}$  和各验证者  $\mathcal{V}$  执行  $[[r]]_2 \leftarrow \text{Random}$ 。随后， $\mathcal{P}$  发送  $y = r - x \in \mathbb{F}_2$  给验证者，所有参与方计算  $[[x]]_2 = [[r]]_2 + y$ 。我们用  $\text{Assign}(x)$  来表示该赋值过程。
- **Shr:** 给定来自  $\mathcal{P}$  的输入  $x \in \mathbb{F}_2^\ell$ ，协议简单地并行调用  $\text{Assign}(x)$   $\ell$  次，以生成  $[[x]]_2$ 。我们用  $\text{Shr}(x)$  来表示此共享过程。请注意，**Shr** 宏仅在密钥生成期间使用，并依赖于可靠的随机源。
- **CheckZero:** 一个认证值  $[[x]]_2$  可以通过检查  $x = 0$  来验证，其方法是让  $\mathcal{P}$  向对应的验证方  $\mathcal{V}_i$  发送  $m_i$ ，然后验证方  $\mathcal{V}_i$  检查是否满足  $m_i = k_i$ 。我们用宏  $\text{CheckZero}([x]_2)$  来表示此验证过程。

在这种情况下，不需要广播，这意味着一个恶意的  $\mathcal{P}$  可能向验证方  $\mathcal{V}_i$  发送正确的  $(m_i)$ ，同时向验证方  $\mathcal{V}_j$  发送错误的  $(m_j)$ ，导致  $m_i = k_i$ ，但  $m_j \neq$



$k_j$ 。这可能会导致  $\mathcal{V}_i$  继续执行，而  $\mathcal{V}_j$  终止。所有验证方可以通过公布其响应并检查一致性来解决此问题。只要至少有一个验证方是诚实的，任何不一致都将被检测到。

### 多验证者乘法验证子协议

该验证子程序在多验证者设置下运行，并利用基于多项式的批量验证技术。此过程的一个功能定义和实例化协议如下所示。

#### $\mathcal{F}_{\text{MV-CheckMULs}}$ 功能

此功能由证明者  $\mathcal{P}$  和  $N$  个验证者  $\mathcal{V}_1, \dots, \mathcal{V}_N$  一起运行，并继承了 PCF（如图 3.8）和  $\mathcal{F}_{\text{MV-ND}}$ （如图 6.1）的所有特性。设  $\mathbb{H} \subset [n]$  为诚实验证者的集合。此外，该功能由以下命令调用。

**检查乘法:** 从  $\mathcal{P}$  和各  $\mathcal{V}$  处接收到  $t$  个乘法三元组  $(\text{CheckMuls}, \text{sid}, \{\llbracket \omega_{\alpha,j} \rrbracket_2, \llbracket \omega_{\beta,j} \rrbracket_2, \llbracket \omega_{\gamma,j} \rrbracket_2\}_{j \in [t]})$  后，其中  $t$  个乘法元组是在多验证者 IT-MACs 中定义的。如果存在某个  $j \in [t]$  使得  $\omega_{\gamma,j} \neq \omega_{\alpha,j} \cdot \omega_{\beta,j}$ ，则设置  $\text{res} = \text{false}$ 。否则，设置  $\text{res} = \text{true}$ 。将  $\{\text{res}_i\}$  发送给敌手，其中  $i \in [N] - \mathbb{H}$ ，且  $\mathcal{V}_i$  的认证共享导致检查失败。对于每个  $i \in \mathbb{H}$ ，等待敌手的输入，并执行如下操作：

- 若输入为  $\text{continue}_i$ ，则将  $\text{res}$  发送给  $\mathcal{V}_i$ 。
- 若输入为  $\text{abort}_i$ ，则向  $\mathcal{V}_i$  发送  $\text{abort}$ 。

#### CheckMuls 协议

CheckMuls 是  $\Pi_{\text{MV-ND}}$  的一个子例程。证明者  $\mathcal{P}$  和  $N$  个验证者  $\mathcal{V}_1, \dots, \mathcal{V}_N$  对  $t$  个乘法三元组  $(\llbracket \omega_{\alpha,i} \rrbracket_2, \llbracket \omega_{\beta,i} \rrbracket_2, \llbracket \omega_{\gamma,i} \rrbracket_2)$  进行一致性检查，其中  $i \in [t]$ 。所有参与方按以下步骤执行：

1. 每个验证者  $\mathcal{V}_i, i \in [N]$  和  $\mathcal{P}$  继承字符串  $\text{str} := \text{H}(\text{mvnd} \parallel \text{sid} \parallel k^* \parallel \{d_j\}_{j \in [t]})^\oplus$ ，运行  $\text{H}(\text{str})$  生成公共随机数  $\chi_1, \dots, \chi_t \in \mathbb{F}_{2^k}$ ，并追加  $\text{str} := \text{str} \parallel \chi_1 \parallel \dots \parallel \chi_t$ 。参与方随机化  $\llbracket z \rrbracket_2 = \sum_{i \in [t]} \chi_i \cdot \llbracket \omega_{\gamma,i} \rrbracket_2$  和  $\llbracket \hat{\omega}_{\alpha,i} \rrbracket_2 = \chi_i \cdot \llbracket \omega_{\alpha,i} \rrbracket_2$ ，对  $i \in [t]$ 。
2. 当  $t > 2$  时：
  - (a) 设置  $t = \frac{t}{2}$ 。所有参与方定义  $t$  个多项式共享值  $\llbracket f_1 \rrbracket_2, \dots, \llbracket f_t \rrbracket_2 \in \mathbb{F}_{2^k}[X]$  和另  $t$  个多项式共享值  $\llbracket g_1 \rrbracket_2, \dots, \llbracket g_t \rrbracket_2 \in \mathbb{F}_{2^k}[X]$ ，使得  $\llbracket f_i(j) \rrbracket_2 = \llbracket \hat{\omega}_{\alpha,j \times t + i} \rrbracket_2, \llbracket g_i(j) \rrbracket_2 = \llbracket \omega_{\beta,j \times t + i} \rrbracket_2$  对于  $j \in \{0, 1\}, i \in \{1, \dots, t\}$ 。
  - (b)  $\mathcal{P}$  将多项式推广为  $h = \sum_{i \in [t]} f_i \cdot g_i \in \mathbb{F}_{2^k}[X]$ 。注意， $h$  的度  $\leq 2$ 。
  - (c) 让  $\{c_0, c_1, c_2\}$  为  $h$  的系数， $\mathcal{P}$  和各  $\mathcal{V}$  执行  $\text{Assign}(c_j)$  并使用  $\llbracket c_j \rrbracket_2, j \in \{0, 1, 2\}$  定义  $\llbracket h \rrbracket_2$ 。（在这里， $\mathcal{P}$  发送的所有通信数据记作  $d_{c,j}$ ，其中  $j \in \{0, 1, 2\}$ ）
  - (d) 参与方运行  $\text{CheckZero}(\sum_{j=1}^2 \llbracket h(j) \rrbracket_2 - \llbracket z \rrbracket_2)$ 。如果此检查失败，参与方输出  $\text{false}$  并中止。
  - (e) 每个验证者  $\mathcal{V}_i$  和  $\mathcal{P}$  追加  $\text{str} := \text{str} \parallel \{d_{c,j}\}_{j \in \{0,1,2\}}$ ，运行  $\text{H}(\text{str})$  生成公共随机数  $\eta \in \mathbb{F}_{2^k}$ ，并将  $\text{str} := \text{str} \parallel \eta$  追加。
  - (f) 所有参与方本地计算  $\llbracket f_1(\eta) \rrbracket_2, \dots, \llbracket f_t(\eta) \rrbracket_2, \llbracket g_1(\eta) \rrbracket_2, \dots, \llbracket g_t(\eta) \rrbracket_2$  和  $\llbracket h(\eta) \rrbracket_2$ 。它们递归返回至 2.(a) 直到  $t \leq 2$ 。

3. 现在  $\mathcal{P}$  和各  $\mathcal{V}$  有最多两个乘法三元组, 记为  $(\llbracket x_i \rrbracket_2, \llbracket y_i \rrbracket_2, \llbracket z_i \rrbracket_2)$ , 对于  $i \in [t]$  和  $t \leq 2$ 。它们进行如下的验证:

(a) 对于  $i \in [t]$ , 所有参与方使用  $\llbracket v_i \rrbracket_2 \leftarrow \text{Random}$  生成认证的随机数并计算

$$\llbracket z_i \rrbracket_2 = \text{Assign}(x_i \cdot y_i), \llbracket \hat{z}_i \rrbracket_2 = \text{Assign}(y_i \cdot v_i)$$

(在这里,  $\mathcal{P}$  发送的所有通信数据记作  $\{d_{z,i}, d_{\hat{z},i}\}_{i \in [t]}$ )

(b) 每个验证者  $\mathcal{V}_i$  和  $\mathcal{P}$  追加  $str := str || \{d_{z,i} || d_{\hat{z},i}\}_{i \in [t]}$ , 运行  $H(str)$  生成公共随机数  $e \in \mathbb{F}_{2^\kappa}$ , 并将  $str := str || e$  追加。

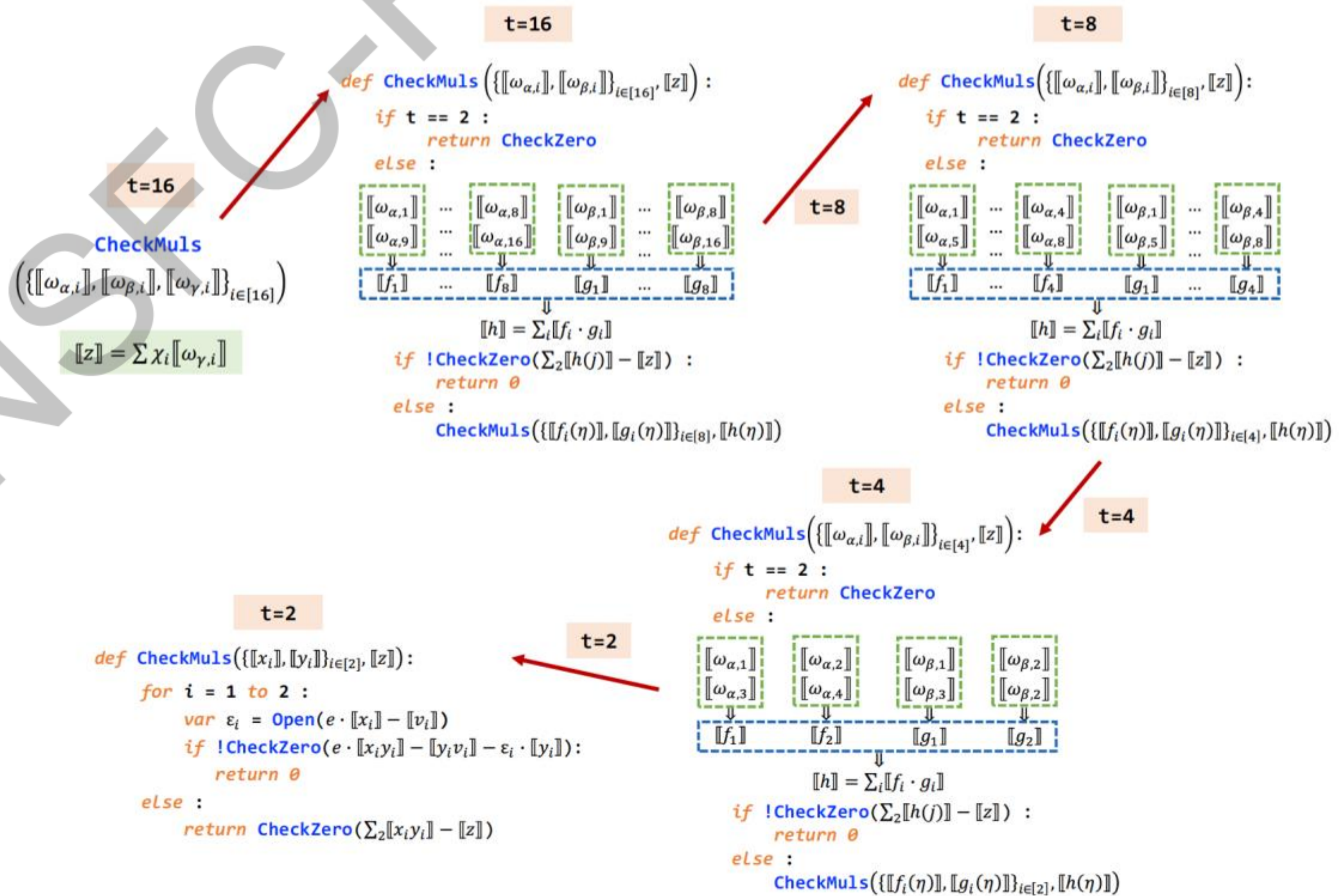
(c) 对于  $i \in [t]$ , 参与方利用  $\llbracket \varepsilon_i \rrbracket_2 = e \cdot \llbracket x_i \rrbracket_2 - \llbracket v_i \rrbracket_2$  公开  $\varepsilon_i$ , 并运行  $\text{CheckZero}(e \cdot \llbracket z_i \rrbracket_2 - \llbracket \hat{z}_i \rrbracket_2 - \varepsilon_i \cdot \llbracket y_i \rrbracket_2)$ 。

(d) 所有参与方运行  $\text{CheckZero}(\sum_{i \in [t]} \llbracket z_i \rrbracket_2 - \llbracket z \rrbracket_2)$ 。如果任何检查失败, 参与方输出 false。否则, 参与方输出 true 并将字符串参数  $str$  传递给  $\Pi_{\text{MV-ND}}$ 。

④ 密钥和交互脚本 (所有通信数据), 即  $\text{sid}, k^*, \{d_j\}_{j \in [t]}$  在  $\Pi_{\text{MV-ND}}$  中定义

**通信与轮次复杂度:** 在检查  $t$  个乘法三元组时, 共需经历  $\log(t)$  轮迭代。在每次迭代中,  $\mathcal{P}$  先在  $\text{Assign}(\cdot)$  宏的执行过程中, 将  $dc, 0, dc, 1, dc, 2 \in \mathbb{F}_{2^\kappa}$  发送给所有  $\mathcal{V}$ 。然后, 在执行  $\text{CheckZero}(\cdot)$  宏时,  $\mathcal{P}$  将 MAC 标签  $mi$  发送给验证者  $\mathcal{V}_i$ 。在整个过程中, 所有验证者仅接收数据, 并在本地执行计算和验证。因此, 每次迭代的通信开销为  $4 \cdot \kappa$ , 并在一个轮次内完成。

下面给出一个当  $t = 2^4$  时,  $\text{CheckMULs}$  的实例





## 多验证者零知识证明框架

项目组基于多验证者IT-MACs设计, 提出多验证者零知识证明框架, 其遵循逐门电路的范式, 其中每条电线上的值形式上在 $N$ 个验证者中被秘密共享为 $[[x]]_2 = \{(x, m_1, \dots, m_N), k_1, \dots, k_N\}$ , 其中每个 $m_i = k_i + x \cdot \Delta_i$ , 且每个 $\mathcal{V}_i$ 持有 $(k_i, \Delta_i), i \in [N]$ 。该值可以通过调用 $PCF$  来生成。正式定义和实例化协议如下:

### $F_{MV-ND}$ 功能

此功能由 $\mathcal{P}$ 和 $\mathcal{V}_1, \dots, \mathcal{V}_N$ 一起运行。该功能由随机数生成电路 $C^* := \{R = \text{PRF}_{dk}(\text{msg}) \cdot G\}$ 参数化, 其中 $G$ 是椭圆曲线群 $\mathbb{G}$ 的生成元, 群的阶为 $q$ 。

1. 当从 $\mathcal{P}$ 接收到 $(\text{mvzk-input}, did, dk)$ , 及从所有验证者 $\mathcal{V}_1, \dots, \mathcal{V}_N$ 接收到 $(\text{mvzk-input}, i)$ , 且具有新的标识符 $sid$ 时, 存储 $(did, dk)$ 。
2. 当从 $\mathcal{P}$ 接收到 $(\text{mvzk-prove}, did, \text{msg})$ , 及从验证者 $\mathcal{V}_1, \dots, \mathcal{V}_N$ 接收到 $(\text{mvzk-verify}, did, \text{msg})$ 时, 若 $(did, dk)$ 已存储, 则设置 $res = \text{true}, R := C^*(dk, \text{msg})$ ; 否则, 设置 $res = \text{false}$ 。
3. 如果 $res = \text{true}$ , 则向各方发送 $(\text{mvzk-proof}, \text{msg}, \llbracket R \rrbracket_q)$ ; 否则, 向各方发送 $\text{abort}$ 。

### $\Pi_{MV-ND}$ 协议

该功能由安全参数 $\kappa$ 和由 $G$ 生成的阶为 $q$ 的椭圆曲线群 $\mathbb{G}$ 参数化。 $n$ 方持有一个带密钥的PRF函数电路 $C := \text{PRF}_{dk}(\text{msg})$ 。电路 $C$ 包含 $t$ 个乘法门, 输入包括秘密共享的派生密钥 $dk \in \mathbb{F}_{2^\kappa}$ 和消息 $\text{msg} \in \{0, 1\}^\ell$ , 并且产生一个秘密共享的非对称值 $r \in \{0, 1\}^\ell$ 作为输出。此外, 所有方就加法电路 $C_{\text{add}} : \{x + y = z\}$ 达成一致, 该电路包含 $t' = |C_{\text{add}}|$ 个乘法门。设 $\tilde{C}$ 为通过 $C$ 后接 $C_{\text{add}}$ 计算得到的电路, 且 $\tilde{t} = t + t'$ 。轮流令每方充当证明者 $\mathcal{P}$ , 其他 $N = n - 1$ 方充当验证者 $\mathcal{V}_1, \dots, \mathcal{V}_N$ 时。

**设置:** 运行一次,  $\mathcal{P}$ 和各 $\mathcal{V}$ 调用 $F_{MV\text{-}eda\text{Bits}}$ 的设置阶段 (见图 5.4)。此外, 从 $\mathcal{P}$ 接收 $\llbracket dk \rrbracket_2$ 。

**证明:** 每个方输入公开已知的消息 $\text{msg}$ :

1. 每个方初始化一个字符串 $str := H(\text{mvnd} || \text{msg} || k^*)$ 。
2. 所有方通过发送 $(eda\text{Bits}, str)$ 给 $F_{MV\text{-}eda\text{Bits}}$ 生成 $\text{mv-edaBits}$ 。

// 对 $\tilde{C}$ 逐门进行评估

3. 对于电路 $\text{PRF}$ , 使用输入 $\llbracket dk \rrbracket_2$ 和 $\text{msg}$ 进行评估, 对于电路 $C_{\text{add}}$ , 使用输入 $\llbracket r \rrbracket_2$ 和 $\llbracket \rho \rrbracket_2$ 进行评估。将 $str$ 初始化为 $\text{msg}$ 。逐门的电路评估过程如下:
  - (a) 按拓扑顺序, 对于每个门 $(\alpha, \beta, \gamma, T) \in \tilde{C}$ , 其输入线值为 $(\omega_\alpha, \omega_\beta)$ 和输出线值为 $\omega_\gamma$ :
    - 如果 $T = \text{ADD}$ ,  $\mathcal{P}$ 和各 $\mathcal{V}$ 本地计算 $\llbracket \omega_\gamma \rrbracket_2 = \llbracket \omega_\alpha \rrbracket_2 + \llbracket \omega_\beta \rrbracket_2$ 。
    - 如果 $T = \text{MULT}$ 且这是第 $j$ 个乘法门,  $\mathcal{P}$ 和各 $\mathcal{V}$ 执行 $\llbracket \omega_\gamma \rrbracket_2 \leftarrow \text{Assign}(\omega_\alpha \cdot \omega_\beta)$ 。

(附加 $str := str || d_j$ , 其中 $d_j$ 是 $\mathcal{P}$ 发送的通信数据)



(b)  $\mathcal{P}$  和各  $\mathcal{V}$  联合检查乘法三元组的正确性, 通过调用

$$\text{CheckMuls}(\{\llbracket \omega_{\alpha,j} \rrbracket_2, \llbracket \omega_{\beta,j} \rrbracket_2, \llbracket \omega_{\gamma,j} \rrbracket_2\}_{j \in [\ell]}).$$

使用种子  $str := H(str)$ 。如果出现任何失败, 双方输出 **false**。否则, 得到  $\llbracket \mathbf{c} \rrbracket_2$ , 满足  $\sum_{j \in [\ell]} 2^{j-1} \cdot \mathbf{c}[j] = \sum_{j \in [\ell]} 2^{j-1} \cdot \mathbf{r}[j] + \sum_{j \in [\ell]} 2^{j-1} \cdot \rho[j] \pmod{q}$  和  $\mathbf{r} = \text{PRF}_{\text{dk}}(\text{msg})$ 。

// 在  $\mathbb{F}_{2^k}$  域和  $\mathbb{G}$  群组之间的转换

4. 所有方公开  $\mathbf{c}$  并计算  $c = \sum_{j \in [\ell]} 2^{j-1} \cdot \mathbf{c}[j]$ 。
5. 如果发生任何失败, 双方输出 **false**。否则, 每个方输出认证后的随机数  $\llbracket R \rrbracket_q = (c - \llbracket \rho \rrbracket_q) \cdot G$  和  $\llbracket r \rrbracket_q = c - \llbracket \rho \rrbracket_q$ 。

### 结果分析:

协议  $MV - ND$  的结构确保在两个方向实例化  $\mathcal{F}_{MV-ND}$  时计算开销最小。当  $\mathcal{P}$  评估电路时, 各  $\mathcal{V}$  处于空闲状态; 而当各  $\mathcal{V}$  验证电路时,  $\mathcal{P}$  无额外任务。

因此, 当某方  $P_i$  在其随机数验证会话中充当  $\mathcal{P}$  时, 在其他方的随机数验证会话期间, 该方在其  $\mathcal{V}_j$  角色中  $j \in [N]$  处于空闲状态, 从而使工作负载在所有方之间均匀分布。

- 逐门评估。每个 AND 门需要一个 VOLE 实例进行单次传输和秘密加法。因此, 每个方在逐门评估阶段主要需要约  $58k$  个 VOLEs。此外, 在 `mv-edaBits` 生成期间评估仿射电路  $C_{\text{aff}}$  及 PRF 电路之后的加法电路  $C_{\text{add}}$  仅引入极小额外 VOLE 开销。
- CheckMuls。使用 Fiat-Shamir 证明技巧, 各方需要对 50.9 KB 消息进行哈希, 并在单次传输中执行  $\log(58k)$  轮计算。每轮计算包括: 对 112 B 消息进行哈希, 在  $\mathbb{F}_{2^k}[X]$  上执行一次二次多项式内积运算, 执行  $3\kappa$  VOLEs, 并进行  $2t$  次一次多项式评估。最终一轮计算需要额外的四个 VOLE 相关运算, 并对 128 B 消息进行哈希。因此, 每个方共需约  $3\kappa \times \log(58k) + 4 \approx 6.1k$  VOLEs, 仅产生轻微额外开销。
- 一致性检查。每个方额外生成最多两个 `mv-edaBits`, 消耗约  $2|q| + 2\ell$  VOLEs, 并执行  $n$  次椭圆曲线乘法和加法, 计算开销可忽略。

综上所述, 每个方的总工作量约为  $65.6k$  VOLEs, 额外开销极小。Boyle et al. 估算了 PCF 评估时间。具体而言, VOLEs 可通过固定密钥 AES 实现, 并可在现代 CPU 上使用 AES-NI 指令优化计算。在 3GHz 处理器上, 单次 PCF 评估约需  $3.57 \times 10^{-3}$  毫秒。基于此, 我们估算在两方设置下进行一次签名计算 (假



设一方受损) 大约需要 230 毫秒。该过程可通过 GPU 线性扩展, 因为 PCF 中的 AES 调用完全独立, 可并行执行。

| 协议    | PRF 使用情况 | 渐近通信量                                   | 具体通信量    | 轮数 | 假设条件       |
|-------|----------|-----------------------------------------|----------|----|------------|
| [166] | 否        | $O( q )$                                | 1.1 KB   | 2  | DDH        |
| [31]  | 否        | $O(m +  q  + \kappa)$                   | 0.88 KB  | 1  | DCR + SRSA |
| [24]  | 是        | $O( C \kappa +  q \kappa)$              | 1.01 MB  | 3  | PRF + CRHF |
| 本文工作  | 是        | $O( C  + \log( C )\kappa + \ell\kappa)$ | 32.47 KB | 2  | LPN        |

对于两方场景, 我们将协议与现有的在恶意环境下用于确定性、无状态且可验证的随机数生成的协议进行了比较, 记录在上表中, 表中  $|C|$  表示伪随机函数 (PRF) 电路的规模,  $\kappa$  是计算安全参数,  $s$  是统计安全参数,  $|q|$  表示  $\mathbb{Z}_q$  中元素的比特长度,  $\ell$  是 PRF 输出的比特长度,  $m$  是文献[31]中使用的一个大参数。具体通信成本是为无状态且确定性地生成  $R = r \cdot G$  计算的, 其中  $r \in \mathbb{Z}_q$  是  $PRF(dk, msg)$  的算术表示。

Nick 等人[166]和 Kondi 等人[31]的工作所实现的通信成本都显著低于其他协议。Kondi 等人[31]的协议实现了最优的一轮执行。然而, 这些方法依赖于用于随机数生成的定制函数, 而非标准通用的 PRF 函数。与我们最为接近的先前工作是发表在美密 2021 上 Garillot 等人[24]的工作, 他们采用 ZKGC 方法来证明  $r = PRF(dk, msg)$ 。我们的随机数生成协议利用了近期基于 LPN 的 PCF 构造, 并且在两方场景下, 与 Garillot 等人的协议相比, 通信成本降低了约 31.9 倍。因此, 我们的工作体现了在效率与 NIST[48]所支持的功能之间的一种权衡。

## (2) 可验证且前向安全的多关键词隐私数据查询

数据安全检索可以为多方神经网络隐私计算协议提供高质量敏感数据源, 动态可搜索对称加密技术能够在支持文件动态增删的同时, 仍然保持对加密数据的可检索能力。然而, 现有的早期动态可搜索对称加密 (DSSE) 方案通常仅支持单关键词查询, 且在半诚实服务器假设下实现前向安全, 这在真实应用场景中难以满足多样化与高安全需求。为解决上述问题, 项目组提出一种支持多关键词合取查询的、具备前向安全性和可验证性的 DSSE 方案 (VFMKQ)。该方案通过链式结构实现前向安全, 并结合 RSA 累加器与同态消息认证码 (HMAC) 确保搜索结果的正确性与完备性。此外, 项目组将正向索引与倒排索引相结合, 以提升

检索效率。

$Setup(\lambda, DB) \rightarrow (K, st; EDB)$ : 该算法由客户端运行, 输入一个安全参数 $\lambda$  和一个原始数据库 $DB$ 。算法输出密钥 $K$ 和数据库初始状态 $st$ 并保存在本地客户端中。此外, 算法还会输出一个加密的数据库 $EDB$ 并保存在云服务器中。

$Update(K, st, (op, ind); EDB) \rightarrow (st'; EDB')$ : 针对文件标识符 $ind$ 及其操作 $op \in \{add, del\}$ , 客户端基于本地存储的密钥 $K$ 和状态 $st$ , 生成相应的更新陷门并更新本地存储的状态 $st'$ 。服务器以加密数据库 $EDB$ 为输入, 输出一个修改后的加密数据库 $EDB'$ 。

$Search(K, st, cq; EDB) \rightarrow (st', R; proof, EDB')$ : 针对一个合取关键词查询 $cq$ , 客户端基于本地存储的密钥 $K$ 和状态 $st$ , 生成相应的搜索陷门并输出一个更新的状态 $st'$ 。服务器以加密的数据库 $EDB$ 为输入, 输出一个修改后的加密数据库 $EDB'$ 、搜索结果 $R$ 以及相应的证明 $proof$ 。

$Verify(K, st, R, proof) \rightarrow true/false$ : 该算法由客户端运行, 输入搜索结果集合 $R$ 和相应证明 $proof$ , 基于本地储存的密钥 $K$ 和状态 $st$ , 输出验证结果 $true/false$ 。

VFMKQ 方案的检索功能主要依赖于倒排索引结构与 EASPE 算法。对于标识符为 $ind$ 、关键词集合为 $W^{ind}$ 的文件 $f$ , 客户端首先生成一个 $n + 1$ 维的索引向量 $I_{ind}$ , 其中前 $n$ 个分量根据对应关键词是否存在 $f$ 中被设为0或1, 最后一个分量固定设为-1。随后, 客户端调用 $EASPE.Enc()$ 算法并结合按位异或操作对 $I_{ind}$ 进行编码, 生成加密索引向量 $I'_{ind}$ 并存储于索引表 $T_2$ , 确保方案的前向安全性。为支持索引的动态更新, 客户端生成一个 PRF 主密钥 $msk$ , 并进一步派生出与关键词 $w$ 对应的加密密钥 $mk[w]$ 。在此基础上, 客户端通过计算 $F(mk[w], Sctr[w])$ 获得关键词 $w$ 的索引密钥 $sk[w]$ 。当客户端对关键词 $w$ 执行一次更新操作时, 使用索引密钥 $sk[w]$ 计算 $Uctr[w]$ 的哈希值, 以确定在 $T_1$ 中的存储地址, 通过异或操作将更新内容 $op \parallel ind \parallel str_{ind}$ 加密后写入该地址。更新操作完成后, 计数器 $Uctr[w]$ 将自增。由于地址的生成仅依赖于客户端掌握的密钥 $sk[w]$ , 在未泄露的前提下, 云服务器无法区分出地址与随机字符串。在搜索阶段, 客户端将最新的索引密钥 $sk[w]$ 提供给云服务器, 用于定位关键词 $w$ 在 $T_1$ 中的相关条目。当云服务器获得与关键词 $w$ 匹配的文件标识集合后, 进一步在 $T_2$ 中解码相应的索引向量, 并通过计算内积来判定最终的匹配结果。完成一次搜索操作后, 计数器 $Sctr[w]$



将自增，客户端也会同步更新并使用新的索引密钥 $sk[w]$ ，确保旧密钥对云服务器失效，实现方案的前向安全性。

VFMKQ 方案的验证机制主要基于 RSA 累加器与 HMAC 技术构建，实现检索结果的正确性与完整性验证。在更新与搜索阶段，客户端分别针对加密索引向量 $\hat{I}_{ind}$ 与查询向量 $I'_{cq}$ ，调用 $HMAC.Auth()$ 算法生成相应验证标签 $\sigma_{ind}$ 和 $\sigma_{cq}$ 。由于 $\sigma_{ind}$ 中的认证值包含 $\hat{I}_{ind}$ 中的每个分量，为避免泄露向量信息，客户端对 $\sigma_{ind}$ 进行按位异或生成密文标签 $\sigma'_{ind}$ 。在接收到云服务器返回的计算结果后，客户端可调用 $HMAC.Verify()$ 算法对其是否正确执行内积运算进行验证，确保检索结果的正确性。在更新阶段，客户端基于当前关键词 $w$ 匹配的文件标识符集合，计算对应的 RSA 累加器值 $Acc(w)$ 并本地存储；在检索阶段，云服务器计算未满足查询条件 $cq$ 中最低频率关键词 $w$ 所对应文件标识符集合的累加器值，作为非包含证明 $proof$ 返回给客户端。客户端基于本地存储的 $Acc(w)$ 、服务器返回的 $proof$ 以及文件标识符集合 $R_1$ ，对搜索结果的正确性进行验证。此外，客户端结合顺序随机挑战机制，验证云服务器是否返回了全部匹配的文档，确保检索结果的完整性。

| Setup 阶段:                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 初始化 RSA 累加器系统参数: $pp \leftarrow Acc.Setup(1^\lambda)$<br>生成 EASPE 陷门生成密钥: $sk_{SPE} \leftarrow EASPE.Setup(1^\lambda)$<br>生成 HMAC 密钥: $sk_{MAC} \leftarrow HMAC.Setup(1^\lambda)$<br>生成系统主密钥: $msk \leftarrow \{0,1\}^\lambda$<br>初始化一个伪随机函数 $F$ 和五个哈希函数 $H_1, H_2, H_3, H_4, H_5$<br>初始化三个字典结构: $Sctr \leftarrow \emptyset, Uctr \leftarrow \emptyset, Vctr \leftarrow \emptyset$<br>初始化两个字典结构: $T_1 \leftarrow \emptyset, T_2 \leftarrow \emptyset$<br>设置密态数据库 $EDB \leftarrow \{T_1, T_2\}$ 将密态数据库 $EDB$ 发送给云服务器。 |

**Setup 阶段:** 客户端首先为 RSA 累加器生成参数 $pp = \{f_1, g, p, q, N\}$ ，并为 EASPE 算法生成密钥 $sk_{SPE} = \{\pi, r_1, r_2, r_3, r_4, r_5, r_6, s, M_1, M_2\}$ 。随后，客户端生成 HMAC 的密钥 $sk_{MAC} = \{K, \alpha\}$ ，并存储伪随机函数  $F$  的主密钥  $msk$ 。客户端初始化一个伪随机函数 $F: \{0,1\}^\lambda \times \{0,1\}^* \rightarrow \{0,1\}^\lambda$ 。同时，客户端初始化五个哈希函数，分别为 $H_1: \{0,1\}^\lambda \times \{0,1\}^* \rightarrow \{0,1\}^\lambda$ 、 $H_2: \{0,1\}^\lambda \times \{0,1\}^* \rightarrow \{0,1\}^\lambda$ 、 $H_3: \{0,1\}^* \rightarrow \{0,1\}^\lambda$ 、 $H_4: \{0,1\}^* \rightarrow \{0,1\}^\lambda$ 以及 $H_5: \{0,1\}^* \rightarrow \{0,1\}^\lambda$ 。为记录每一个关键词对应状态，客户端维护三个关键词状态字典 $Sctr$ 、 $Uctr$ 和 $Vctr$ ，其中 $Sctr[w]$ 表示关键词



$w$ 当前的检索次数,客户端可利用 $Sctr[w]$ 与主密钥 $msk$ 派生倒排索引的加密密钥; $Uctr[w]$ 表示关键词 $w$ 的更新次数,用于生成关键词在索引中的地址和值; $Vctr[w]$ 表示当前匹配关键词 $w$ 的文件数(即已添加但未被删除的文件数量)。最后,客户端生成加密数据库 $EDB$ ,内部包含两个字典映射结构: $T_1$ 和 $T_2$ ,其中 $T_1$ 存储倒排索引信息,即关键词到文档标识符加密索引的映射; $T_2$ 存储正排索引及其认证标签,即文档标识符到关键词加密索引的映射

**Update 阶段:** 对于每一个更新的文件标识符 $ind$ ,客户端随机生成一个字符串 $str_{ind}$ ,用于加密正排索引以确保前向安全性。随后,客户端初始化一个 $n+1$ 维的向量 $I_{ind}$ ,其中对应关键词 $w \in W^{ind}$ 的位置分量被赋值为1,其余位置赋值为0,第 $n+1$ 维位置的分量赋值为-1。客户端利用 $EASPE.Enc()$ 算法对 $I_{ind}$ 进行加密,得到密文向量 $\hat{I}_{ind}$ 。为确保正排索引的可验证性,客户端调用 $HMAC.Auth()$ 算法生成对应的认证标签 $\sigma_{ind}$ 。同时,客户端使用预先生成的随机字符串 $str_{ind}$ 对 $\hat{I}_{ind}$ 和 $\sigma_{ind}$ 分别进行异或操作,防止之前搜索陷门匹配已更新的关键词密文索引。

对于每个关键词 $w \in W^{ind}$ ,客户端首先判断 $Uctr[w]$ 是否为空,若为空则表明该关键词首次被更新,需分别初始化 $Uctr[w]$ 、 $Sctr[w]$ 及 $Vctr[w]$ 。当关键词 $w$ 被更新时, $Uctr[w]$ 会自增。客户端根据主密钥 $msk$ 与关键词 $w$ 生成密钥 $mk[w]$ ,基于 $mk[w]$ 及搜索次数 $Sctr[w]$ 生成索引密钥 $sk[w]$ ,该密钥用于依据更新次数 $Uctr[w]$ 计算倒排索引中对应的地址 $addr[w]$ 和元素值 $value[w]$ 。倒排索引生成后,客户端利用RAS累加器生成相应的证明以验证搜索结果的完整性与正确性。若操作为添加文件, $Vctr[w]$ 自增1。当 $Uctr[w] = 1$ 时,表明关键词 $w$ 首次被更新,此时累加器的值设定为 $Acc(w) = g^{prime(H_5(ind, H_5(\hat{I}_{ind})))} \bmod N$ ;否则累加器的值更新为 $Acc(w) = Acc(w)^{prime(H_5(ind, H_5(\hat{I}_{ind})))} \bmod N$ 。若操作为删除文件, $Vctr[w]$ 自减1,且累加器的值更新为 $Acc(w) = Acc(w)^{prime(H_5(ind, H_5(\hat{I}_{ind})))^{-1}} \bmod N$ 。这里假设被删除的文件为之前已添加的文件。最后,客户端将字典列表 $T_1$ 和 $T_2$ 传输至云服务器,客户端本地保存关键词状态列表 $Uctr$ 、 $Sctr$ 、 $Vctr$ 以及对应的累加器证明值 $Acc$ 。

| Update 阶段                                       |
|-------------------------------------------------|
| 初始化随机字符串 $str_{ind} \leftarrow \{0,1\}^\lambda$ |
| 如果 $op = add$                                   |



对于每一个  $w_i \in W^{ind}$ , 设置  $I_{ind}[i] \leftarrow 1$   
 计算  $I_{ind}[n+1] \leftarrow -1$   
 计算加密向量  $\hat{I}_{ind} \leftarrow \text{EASPE.Enc}(sk_{SPE}, I_{ind})$   
 生成标签  $\sigma_{ind} \leftarrow \text{HMAC.Auth}(sk_{MAC}, L_{\hat{I}_{ind}}, \hat{I}_{ind})$   
 如果  $i = 1$  到  $i = 2n + 8$   
 设置  $I'_{ind}[i] \leftarrow \hat{I}_{ind}[i] \oplus H_3(str_{ind} \parallel i)$   
 设置  $\sigma'_{ind}[i][1] \leftarrow \sigma_{ind}[i][1] \oplus H_3(str_{ind} \parallel i)$   
 设置  $\sigma'_{ind}[i][2] \leftarrow \sigma_{ind}[i][2] \oplus H_4(str_{ind} \parallel i)$   
 设置  $T_2[ind] \leftarrow (I'_{ind}, \sigma'_{ind})$   
 对于  $w_i \in W^{ind}$   
 设置  $Uctr[w] \leftarrow 0, Sctr[w] \leftarrow 1, Vctr[w] \leftarrow 0$   
 设置  $Uctr[w] \leftarrow Uctr[w] + 1$   
 设置  $mk[w] \leftarrow F(msk, w)$   
 设置  $sk[w] \leftarrow F(mk[w], Sctr[w])$   
 设置  $addr[w] \leftarrow H_1(sk[w], Uctr[w])$   
 设置  $value[w] \leftarrow H_2(sk[w], Uctr[w]) \oplus (op \parallel ind \parallel str_{ind})$   
 如果  $op = add$ :  $Vctr[w] \leftarrow Vctr[w] + 1$   
 如果  $Uctr[w] = 1$ :  $Acc(w) \leftarrow g^{\text{prime}(H_5(ind, H_5(\hat{I}_{ind})))} \bmod N$   
 否则: 设置  $Acc(w) \leftarrow Acc(w)^{\text{prime}(H_5(ind, H_5(\hat{I}_{ind})))} \bmod N$   
 如果  $op = add$   
 设置  $Acc(w) \leftarrow Acc(w)^{\text{prime}(H_5(ind, H_5(\hat{I}_{ind})))^{-1}} \bmod N$   
 设置  $Vctr[w] \leftarrow Vctr[w] - 1$   
 设置  $T_1[addr[w]] \leftarrow value[w]$   
 客户端上传  $T_1$  与  $T_2$  到云服务器, 并在本地安全存储  $Acc, Uctr, Sctr, Vctr$

**Search 阶段:** VFMKQ 搜索过程包含陷门生成和检索索引两个阶段。

**生成搜索陷门阶段:** 对于合取查询  $cq$  中的每个关键词  $w$ , 客户端首先在  $Vctr$  中找到最小的  $Vctr[w]$  (表示与关键词  $w$  匹配的文件数量最少)。同时, 客户端获取该关键词对应的搜索次数  $Sctr[w]$  和更新次数  $Uctr[w]$ 。利用主密钥  $msk$  和关键词  $w$ , 客户端生成密钥  $mk[w] = F(msk, w)$ 。随后, 客户端基于搜索次数  $Sctr[w]$



生成当前搜索陷门 $sk[w]$ 及前一次搜索陷门 $sk'[w]$ 。客户端构造搜索向量 $I_{cq}$ ，其中对应于查询关键词 $w \in cq$ 的位置分量设置为 1，其余位置分量设置为 0，且最后一个分量设置为 $m+r$ ， $r$ 是客户端选取的随机数。客户端利用算法  $EASPE.Trapdoor()$  对查询向量 $I_{cq}$ 进行加密，得到加密后的向量 $I'_{cq}$ ，使用算法  $HMAC.Auth()$  生成对应的认证标签 $\sigma_{cq}$ 。最终，客户端将搜索陷门 $tk = \{sk[w], sk'[w], Uctr[w], I'_{cq}, \sigma_{cq}, r\}$ 发送至云服务器。

**检索索引阶段：**服务器首先判断本次对关键词 $w$ 的搜索是否为首次搜索。若为首次搜索，服务器将关键词 $w$ 的上次更新计数初始化为 $CST[w] = 1$ ，同时将上次搜索结果置为空集 $R_1 \leftarrow \emptyset$ ，证明值初始化为群元素 $proof = g$ ；若非首次搜索，服务器则从缓存集合 $CDB[sk'[w]]$ 中获取上次返回的搜索结果 $R_1$ 和对应证明 $proof$ 。随后，服务器根据本次搜索陷门中的更新计数 $Uctr[w]$ 与上次记录的更新计数 $CST[w]$ ，依次生成对应地址 $addr[w]$ 和元素值 $value[w]$ ，其中包含文件操作类型 $op$ 、文件标识符 $ind$ 以及对应的随机字符串 $str_{ind}$ 。若 $op = add$ ，服务器将文件标识符 $ind$ 加入添加集合 $ID_{add}$ ；若 $op = del$ ，服务器将 $ind$ 加入删除集合 $ID_{del}$ 。由此，服务器可得到当前匹配的文件集合 $ID_{res} = ID_{add} - ID_{del}$ 。对于集合 $ID_{del}$ 。

| 生成搜索陷门阶段                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>对于每个<math>w_i \in cq</math>，</p> <p>客户端首先在<math>Vctr</math>中找到最小的<math>Vctr[w]</math>对应的搜索次数<math>Sctr[w]</math>和更新次<math>Uctr[w]</math></p> <p>设置<math>mk[w] \leftarrow F(msk, w)</math></p> <p>如果<math>Sctr[w] = 0</math>：设置<math>sk'[w] \leftarrow \emptyset</math></p> <p>否则：设置<math>sk'[w] \leftarrow F(mk[w], Sctr[w] - 1)</math></p> <p><math>sk[w] \leftarrow F(mk[w], Sctr[w])</math></p> <p>初始化向量 <math>I_{cq}</math>，所有元素置为 0</p> <p>对于每个<math>w_i \in cq</math>，令<math>I_{cq}[i] = 1</math></p> <p>随机选择随机数<math>r</math>，设置<math>I_{cq}[n+1] = m+r</math></p> <p>计算陷门<math>I'_{cq} = EASPE.TrapGen(sk_{SPE}, I_{cq})</math></p> <p>生成标签<math>\sigma_{cq} = HMAC.Auth(sk_{MAC}, L_{I'_{cq}}, I'_{cq})</math></p> <p>发送陷门<math>tk = \{sk[w], sk'[w], Uctr[w], I'_{cq}, \sigma_{cq}, r\}</math>到云服务器</p> |



由此，服务器可得到当前匹配的文件集合  $ID_{res} = ID_{add} - ID_{del}$ 。对于集合  $ID_{res}$  中的每个文件标识符  $ind$ ，服务器从  $T_2$  中获取加密索引  $I'_{ind}$  和加密认证标签  $\sigma'_{ind}$ ，并利用先前获得的随机字符串  $str_{ind}$  对其进行解密，恢复出索引  $\hat{I}_{ind}$  和认证标签  $\sigma_{ind}$ 。随后，服务器调用  $EASPE.Query()$  算法验证该文件对应的索引向量是否匹配查询陷门。若两向量的内积等于指定值  $r$ ，服务器利用  $HMAC.Eval()$  算法计算标签  $\sigma$ ，并将文件标识符  $ind$ 、认证标签  $\sigma$  及  $H_5(\hat{I}_{ind})$  加入结果集  $R_1$  中；否则，服务器利用 RSA 累加器计算证明值  $proof$  并更新证明结果。随后，服务器将本次搜索结果  $R_1$  及证明  $proof$  缓存至  $CDB[sk[w]]$ 。最后，云服务器将  $R_1$  和  $proof$  返回给客户端，客户端将搜索计数器更新为  $Sctr[w] = Sctr[w] + 1$ 。

| 检索索引阶段                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>如果 <math>sk'[w] \neq \emptyset</math></p> <p>设置 <math>CST[w] \leftarrow 1</math></p> <p>设置 <math>R_1 \leftarrow \emptyset</math></p> <p>设置 <math>proof \leftarrow g</math></p> <p>否则</p> <p>设置 <math>(R_1, proof) \leftarrow CDB[sk'[w]]</math></p> <p>对于 <math>uctr = CST[w]</math> 到 <math>Uctr[w]</math></p> <p>设置 <math>addr[w] \leftarrow H_1(sk[w], uctr)</math></p> <p>设置 <math>value[w] \leftarrow T_1[addr[w]]</math></p> <p>设置 <math>op \parallel ind \parallel str_{ind} \leftarrow H_2(sk[w], uctr) \oplus value[w]</math></p> <p>如果 <math>op = add</math></p> <p>设置 <math>ID_{add} \leftarrow ID_{add} \cup \{ind\}</math></p> <p>否则</p> <p>设置 <math>ID_{del} \leftarrow ID_{del} \cup ind</math></p> <p>设置 <math>ID_{res} \leftarrow ID_{add} - ID_{del}</math></p> <p>设置 <math>CST[w] \leftarrow Uctr[w]</math></p> <p>对于 <math>ind \in ID_{res}</math></p> <p>设置 <math>(I'_{ind}, \sigma'_{ind}) \leftarrow T_2[ind]</math></p> <p>对于 <math>i=1</math> 到 <math>i=2n+8</math></p> |



设置  $\hat{I}_{ind}[i] \leftarrow I'_{ind}[i] \oplus H_3(str_{ind} \parallel i)$   
 设置  $\sigma_{ind}[i][1] \leftarrow \sigma'_{ind}[i][1] \oplus H_3(str_{ind} \parallel i)$   
 设置  $\sigma_{ind}[i][2] \leftarrow \sigma'_{ind}[i][2] \oplus H_4(str_{ind} \parallel i)$   
 如果乘积为  $r$   
 计算  $\sigma \leftarrow HMAC.Eval(f_2, \sigma_{ind}, \sigma_{cq})$   
 计算  $R_1 \leftarrow R_1 \cup \{ind, \sigma, H_5(\hat{I}_{ind})\}$   
 否则  
 计算  $proof \leftarrow proof^{\text{prime}(H_5(ind, H_5(\hat{I}_{ind})))} \bmod N$   
 设置  $CDB[sk[w]] \leftarrow (R_1, proof)$   
 设置  $Sctr[w] \leftarrow Sctr[w] + 1$

**Verify 阶段:** 客户端在接收到搜索结果和证明后, 对于每个文件标识符  $ind$ , 首先利用  $HMAC.Verify()$  算法验证对应的两个等式是否成立。此外, 客户端利用 RSA 累加器验证返回的文件标识符是否属于匹配最低频次关键词  $w$  的集合, 以及索引向量与文件标识符之间的对应关系。若上述验证均通过, 则表明返回结果是正确的。验证等式  $Acc(w) = proof$ 。

| Verify 阶段                                                                                                                                                                                                                                      |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 对于每一个结果<br>将 $\sigma$ 解析为 $(y_0, y_1, y_2)$<br>验证 $y_0 = r$<br>验证 $f_2(L_{\hat{I}_{ind}}, L_{I'_{cq}}) = \sum_{k=0}^2 y_k \alpha^k$<br>计算 $proof \leftarrow proof^{\text{prime}(H_5(ind, H_5(\hat{I}_{ind})))} \bmod N$<br>验证 $Acc(w) = proof$ |

VFMKQ 方案可验证性包括正确性和完整性两个方面, 其中 RSA 累加器和 HMAC 算法能够保证检索结果的正确性, 但无法完全保证检索结果的完整性。针对完整性验证问题, 本章引入了一种基于随机顺序挑战技术以验证检索结果的完整性。具体而言, 云服务器首先检索最低频次关键词对应的倒排索引, 获得文件集合  $ID_{res}$ 。随后, 云服务器对集合  $ID_{res}$  中的所有文件执行  $EASPE.Query()$  算法, 得到匹配结果并根据相似度得分进行降序排序。假设集合  $ID_{res}$  中元素数量为  $M$ , 则排序的计算复杂度为  $O(M \log M)$ 。云服务器将排序后的匹配结果列表及相



应证明发送给客户端。客户端随机选择 $t$ 个文件进行挑战，云服务器返回对应的计算结果和验证标签。客户端验证所有计算结果是否均正确且小于阈值 $r$ ，并验证计算结果的排序是否正确。当所有验证通过时，客户端认定返回的检索结果是完整的。若云服务器错误地返回了比例为 $q$ 的文件，则客户端在挑战 $t$ 个文件时，检测出错误结果的概率为 $1 - \frac{(qM)!}{t!}$ 。

### (3) 一种具有特权方的恶意敌手模型多方隐私保护计算协议

一种具有特权方的恶意敌手模型多方计算框架 pMPL 对参与方的权力进行了区分，分为特权方与辅助方两种角色。特权方是计算任务的发起者，不会掉线，不会与其他参与方合谋，并且将获得最终的模型；辅助方则主要用于提供更多的数据，可能出现掉线、共谋等情况。因此，pMPL 可以用于处理较为复杂的场景。例如，在金融风险控制的场景中，金融专业公司希望与拥有大量用户数据的多家互联网服务提供商合作，通过对其提供的用于训练的数据支付费用，训练一种用于金融风控的准确模型。然而，由于互联网服务提供商可能由于系统或网络故障，或者出于商业目的有意退出计算过程。采用 pMPL 可以容忍某个辅助方（互联网服务提供商）的退出，并正常完成安全计算。

本项目设计了一套具有特权方的恶意敌手模型多方隐私保护计算协议，在存在一个特权方与两个辅助方共同参加计算的情况下，即便某个辅助方合谋或者掉线，剩下的参与方也能够完成安全计算，兼顾隐私性、高效性和鲁棒性。

以下对本方案的描述中，将使用秘密分享协议 $\Pi_{shr}$ 实现向量空间秘密共享，即数据持有方 $P_i$ 对为数据 $x$ 构造一个三维向量 $\mathbf{s} = (x, s_1, s_2)^T$ ，其中 $s_1, s_2$ 均为随机数，计算 $\langle x \rangle_j = \varphi(j) \cdot \mathbf{s}, j \in \{0, 1, 2, 3\}$ ，其中， $\varphi(0) = (1, 0, 1), \varphi(1) = (1, 1, 2^l - 1), \varphi(2) = (2, 2, 2^l - 3), \varphi(3) = (3, 3, 2^l - 4)$ ，取 $l = 64$ ，并使 $P_0$ 得到 $\langle x \rangle_0, \langle x \rangle_3$ ， $P_1$ 持有 $\langle x \rangle_1$ ， $P_2$ 持有 $\langle x \rangle_2$ 。使用秘密重构协议 $\Pi_{rec}$ 实现对向量空间秘密共享份额的重构，即参与方通过交换份额，计算 $x = c_0 \cdot \langle x \rangle_0 + c_1 \cdot \langle x \rangle_1 + c_2 \cdot \langle x \rangle_2$ 或 $x = c_0' \cdot \langle x \rangle_0 + c_1' \cdot \langle x \rangle_1 + c_3' \cdot \langle x \rangle_3$ 或 $x = c_0'' \cdot \langle x \rangle_0 + c_2'' \cdot \langle x \rangle_2 + c_3'' \cdot \langle x \rangle_3$ ，重构出 $x$ 的真实值。使用安全乘法协议 $\Pi_{mul}$ 实现秘密份额形式的两个数的相乘，得到的乘法结果均为秘密份额形式，即对于 $P_0$ 输入的 $\langle x \rangle_i, \langle y \rangle_i, i \in \{0, 3\}$ ， $P_1$ 输入的 $\langle x \rangle_1, \langle y \rangle_1$ ， $P_2$ 输入的 $\langle x \rangle_2, \langle y \rangle_2$ ，分别为三方输出 $\langle z \rangle_0 = \langle x \cdot y \rangle_0$ 与 $\langle z \rangle_3 = \langle x \cdot y \rangle_3, \langle z \rangle_1 = \langle x \cdot y \rangle_1$ 、



$\langle z \rangle_2 = \langle x \cdot y \rangle_2$ 。使用  $Gen_{\alpha, \beta}^{\leq} \rightarrow (k_0, k_1)$  实现函数秘密共享的密钥生成；使用  $Eval_{\alpha, \beta}^{\leq}(i, k_i, x) \rightarrow [f]_i$  实现函数共享秘密份额的生成，其中，当  $x < \alpha$  时， $[f]_0 + [f]_1 = f = \beta$ ，否则  $[f]_0 + [f]_1 = f = 0$ 。

在本方案中，参与方集合为  $\mathcal{P} = \{P_0, P_1, P_2\}$ ，其中  $P_0$  是不掉线、不共谋、作为训练任务发起者的特权方， $P_1, P_2$  是可能掉线、合谋的且不会得到最终训练模型的辅助方。本方案包含三个基础运算协议：安全除法协议、安全乘法的扩展协议以及安全比较协议。

### 1) 安全除法协议：

特权方  $P_0$  持有秘密份额  $\langle x \rangle_0, \langle y \rangle_0, \langle x \rangle_3, \langle y \rangle_3$ ，辅助方  $P_1$  持有秘密份额  $\langle x \rangle_1, \langle y \rangle_1$ ，辅助方  $P_2$  持有秘密份额  $\langle x \rangle_2, \langle y \rangle_2$ 。通过共同参与执行安全除法协议，各个参与方  $P_i$  分别得到除法结果的秘密份额  $\langle z \rangle_i = \langle x/y \rangle_i, i \in \{0, 1, 2\}$ ，此外，特权方  $P_0$  额外得到份额  $\langle z \rangle_3 = \langle x/y \rangle_3$ 。

该协议预处理阶段包括以下步骤：

1.  $P_i$  分别生成一个随机数  $r_i$ ；
2.  $P_i$  分别执行秘密分享协议  $\Pi_{shr}(P_i, r_i)$ ，因此， $P_0$  持有  $\langle r_i \rangle_0, \langle r_i \rangle_3$ ， $P_1$  持有  $\langle r_i \rangle_1$ ， $P_2$  持有  $\langle r_i \rangle_2$ ，其中  $i \in \{0, 1, 2\}$ 。
3.  $P_i$  分别本地计算  $\langle r \rangle_i = \langle r_0 \rangle_i + \langle r_1 \rangle_i + \langle r_2 \rangle_i, i \in \{0, 1, 2\}$ ， $P_0$  额外计算  $\langle r \rangle_3 = \langle r_0 \rangle_3 + \langle r_1 \rangle_3 + \langle r_2 \rangle_3$ 。

预处理完成后，该协议中各参与方的交互过程如错误!未找到引用源。所示，具体包括以下步骤：

1. 参与方共同执行安全乘法协议  $\Pi_{mul}(\mathcal{P}, \langle r \rangle, \langle y \rangle)$ ， $P_i$  得到  $\langle r \cdot y \rangle_i, i \in \{0, 1, 2\}$ ， $P_0$  额外得到份额  $\langle r \cdot y \rangle_3$ ；
2. 参与方共同执行重构协议  $\Pi_{rec}(\mathcal{P}, \langle r \cdot y \rangle)$ ，均得到  $r \cdot y$ ；
3. 参与方共同执行安全乘法协议  $\Pi_{mul}(\mathcal{P}, \langle r \rangle, \langle x \rangle)$ ， $P_i$  得到  $\langle r \cdot x \rangle_i, i \in \{0, 1, 2\}$ ， $P_0$  额外得到份额  $\langle r \cdot x \rangle_3$ ；
4.  $P_i$  分别本地计算  $\langle z \rangle_i = \langle r \cdot x \rangle_i / r \cdot y, i \in \{0, 1, 2\}$ ， $P_0$  额外计算  $\langle z \rangle_3 = \langle r \cdot x \rangle_3 / r \cdot y$ 。

### 2) 安全乘法的扩展协议：

特权方  $P_0$  持有  $x$  与秘密份额  $\langle y \rangle_0, \langle y \rangle_3$ ，辅助方  $P_1$  持有秘密份额  $\langle y \rangle_1$ ，辅助方  $P_2$



持有秘密份额 $\langle y \rangle_2$ 。通过共同参与执行安全乘法的扩展协议，各个参与方 $P_i$ 分别得到乘法结果的秘密份额 $\langle z \rangle_i = \langle x \cdot y \rangle_i, i \in \{0,1,2\}$ ，此外，特权方 $P_0$ 额外得到份额 $\langle z \rangle_3 = \langle x \cdot y \rangle_3$ 。

该协议预处理阶段包括以下步骤：

1. 参与方执行向量乘法三元组生成协议 $\Pi_{vmtgen}(\mathcal{P})$ ，因此， $P_0$ 持有 $\langle u \rangle_0, \langle v \rangle_0, \langle h \rangle_0$ 与 $\langle u \rangle_3, \langle v \rangle_3, \langle h \rangle_3$ ， $P_1$ 持有 $\langle u \rangle_1, \langle v \rangle_1, \langle h \rangle_1$ ， $P_2$ 持有 $\langle u \rangle_2, \langle v \rangle_2, \langle h \rangle_2$ ，其中 $h = u \cdot v$ 。

预处理完成后，该协议中各参与方的交互过程如[错误!未找到引用源。](#)所示，具体包括以下步骤：

1. 如果不存在辅助方掉线， $P_0$ 收到来自 $P_i$ 的 $\langle u \rangle_i, i \in \{1,2\}$ ，并通过计算 $u = c_0 \cdot \langle u \rangle_0 + c_1 \cdot \langle u \rangle_1 + c_2 \cdot \langle u \rangle_2$ 重构出 $u$ 的真实值；如果存在某个辅助方（以 $P_2$ 为例）掉线， $P_0$ 收到来自 $P_1$ 的 $\langle u \rangle_1$ ，并通过计算 $u = c'_0 \cdot \langle u \rangle_0 + c'_1 \cdot \langle u \rangle_1 + c'_3 \cdot \langle u \rangle_3$ 重构出 $u$ 的真实值；

2.  $P_0$ 本地计算 $e = x + u$ ，并将 $e$ 发送给辅助方 $P_i, i \in \{1,2\}$ ；

3.  $P_i$ 分别本地计算 $\langle d \rangle_i = \langle y \rangle_i + \langle v \rangle_i, i \in \{0,1,2\}$ ， $P_0$ 额外计算 $\langle d \rangle_3 = \langle y \rangle_3 + \langle v \rangle_3$ ；

4. 参与方共同执行 $\Pi_{rec}(\mathcal{P}, \langle d \rangle)$ ，均得到 $d = y + v$ ；

5.  $P_i$ 分别本地计算 $\langle z \rangle_i = \langle h \rangle_i - \langle v \rangle_i \cdot e, i \in \{1,2\}$ 。同时， $P_0$ 本地计算 $\langle z \rangle_0 = x \cdot d/c_0 + \langle h \rangle_0 - \langle v \rangle_0 \cdot e$ 与 $\langle z \rangle_3 = \langle h \rangle_3 - \langle v \rangle_3 \cdot e$ 。

### 3) 安全比较协议：

特权方 $P_0$ 持有秘密份额 $\langle x \rangle_0, \langle y \rangle_0, \langle x \rangle_3, \langle y \rangle_3$ ，辅助方 $P_1$ 持有秘密份额 $\langle y \rangle_1$ ，辅助方 $P_2$ 持有秘密份额 $\langle y \rangle_2$ 。通过共同参与执行安全比较协议，各个参与方 $P_i$ 分别得到比较结果的秘密份额 $\langle z \rangle_i = \langle \mathbf{1}\{x > y\} \rangle_i, i \in \{0,1,2\}$ ，此外，特权方 $P_0$ 额外得到份额 $\langle z \rangle_3 = \langle \mathbf{1}\{x > y\} \rangle_3$ 。

在不存在辅助方掉线的情况下，该协议中各参与方的交互过程如[错误!未找到引用源。](#)所示，具体包括以下步骤：

1.  $P_0$ 执行 $Gen_{N/2,1}^< \rightarrow (k_1, k_2)$ 生成密钥，并将密钥 $k_i$ 分别发送给 $P_i$ ，其中 $i \in \{1,2\}$ ；

2.  $P_0$ 生成一个随机数 $r$ ；



3.  $P_i$  分别本地计算  $\langle x - y \rangle_i = \langle x \rangle_i - \langle y \rangle_i, i \in \{0,1,2\}$ ;
4. 参与方共同执行  $\Pi_{extmul}(\mathcal{P}, r, \langle x - y \rangle)$ , 使  $P_0$  得到  $\langle r \cdot (x - y) \rangle_0, \langle r \cdot (x - y) \rangle_3$ ,  $P_1$  得到  $\langle r \cdot (x - y) \rangle_1$ ,  $P_2$  得到  $\langle r \cdot (x - y) \rangle_2$ ;
5.  $P_0$  将  $\langle r \cdot (x - y) \rangle_0$  发送给  $P_1$  和  $P_2$ ,  $P_1$  和  $P_2$  相互发送  $\langle r \cdot (x - y) \rangle_i, i \in \{1,2\}$ ;
6.  $P_1$  和  $P_2$  计算  $r \cdot (x - y) = c_0 \cdot \langle r \cdot (x - y) \rangle_0 + c_1 \cdot \langle r \cdot (x - y) \rangle_1 + c_2 \cdot \langle r \cdot (x - y) \rangle_2$ , 均得到  $r \cdot (x - y)$  的值;
7. 辅助方  $P_i$  分别执行  $Eval_{N/2,1}^{\leq} \rightarrow (i, k_i, r \cdot (x - y)) \rightarrow [f]_i$ , 得到  $[f]_i$ , 其中  $i \in \{1,2\}$ ;
8. 辅助方  $P_i$  通过交换  $[f]_i, i \in \{1,2\}$ , 计算  $f = [f]_1 + [f]_2$  得到  $f$  值, 并执行  $\Pi_{shr}(P_i, f)$ , 使  $P_0$  得到  $\langle f \rangle_0, \langle f \rangle_3$ ,  $P_1$  得到  $\langle f \rangle_1$ ,  $P_2$  得到  $\langle f \rangle_2$ ;
9.  $P_0$  计算  $\langle z \rangle_0 = \langle f \rangle_0 \oplus MSB(r), \langle z \rangle_3 = \langle f \rangle_3$ ,  $P_i$  计算  $\langle z \rangle_i = \langle f \rangle_i, i \in \{1,2\}$ 。
- 10.

在存在辅助方掉线（以  $P_2$  掉线为例）的情况下，该协议具体步骤如下：

1.  $P_i$  分别生成随机数  $r_i, i \in \{0,1\}$ ;
2.  $P_0$  和  $P_1$  借助现有的函数秘密共享技术，在相互不泄露敏感信息的前提下，通过  $Gen_{N/2,1}^{\leq} \rightarrow (k_0, k_1)$  分别得到  $k_0, k_1$ ;
3.  $P_i$  分别本地计算  $\langle y \rangle_i - \langle x \rangle_i + r_i/c_i', i \in \{0,1\}$ ,  $P_0$  额外计算  $\langle y \rangle_3 - \langle x \rangle_3$ ;
4.  $P_0$  和  $P_1$  交换秘密份额，并计算  $c_0' \cdot (\langle y \rangle_0 - \langle x \rangle_0 + r_0/c_0') + c_1' \cdot (\langle y \rangle_1 - \langle x \rangle_1 + r_1/c_1') + c_3' \cdot (\langle y \rangle_3 - \langle x \rangle_3)$ , 重构出  $y - x + r_0 + r_1$ ;
5.  $P_i$  分别执行  $Eval_{r_0+r_1,1}^{\leq} \rightarrow (i, k_i, y - x + r_0 + r_1) \rightarrow [f]_i$ , 其中  $i \in \{0,1\}$ ;
6.  $P_i$  分别执行  $\Pi_{shr}(P_i, [f]_i)$ , 使  $P_0$  得到  $\langle [f]_i \rangle_0, \langle [f]_i \rangle_3$ ,  $P_1$  得到  $\langle [f]_i \rangle_1$ ,  $P_2$  得到  $\langle [f]_i \rangle_2$ , 其中  $i \in \{0,1\}$ ;
7.  $P_i$  分别本地计算  $\langle z \rangle_i = \langle [f]_0 \rangle_i + \langle [f]_1 \rangle_i, i \in \{0,1,2\}$ ,  $P_0$  额外计算  $\langle z \rangle_3 = \langle [f]_0 \rangle_3 + \langle [f]_1 \rangle_3$ 。

本文所提出的方案在以下实验环境中进行验证。本文在三个 Linux 服务器上进行所有的实验，其中每个服务器都配置有 2.3 GHz 的 Intel(R) Xeon(R) Processor (Skylake, IBRS), RAM 是 8GB, 运行操作系统为 Ubuntu 18.04, 这三个服务器分别代表特权方  $P_0$ 、辅助方  $P_1$  和辅助方  $P_2$ 。此外，本文将模拟两种网络



环境：LAN（带宽 2GBps，时延 0.3ms）与 WAN（带宽 40MBps，时延 40ms）。所采用的数据集是 UCI 数据集。

实验过程包括两个部分。第一部分是对所提出的安全基础运算协议进行运算开销与通信开销的评估和比较。实验结果表明，与最近的类似协议相比，本方案中的安全除法协议执行速度提升至少 93.5 倍，通信开销减少 90%；安全乘法协议执行速度提升约 1.5 倍，通信开销减少约 20%；安全比较协议在适配更复杂场景的同时保持相似效率。

第二部分是对由基础协议组合实现的多方隐私保护决策树训练 SecureCART 方案进行性能评估与比较。对于样本数量为 50 的数据集，本方案的训练时间约为 0.41 秒；对于样本数量为 5000 的数据集，本方案的训练时间约为 3.29 秒。此外，对于不同数据集，本方案的预测准确度均保持在 85%至 97%之间，精确度损失在可接受范围内。

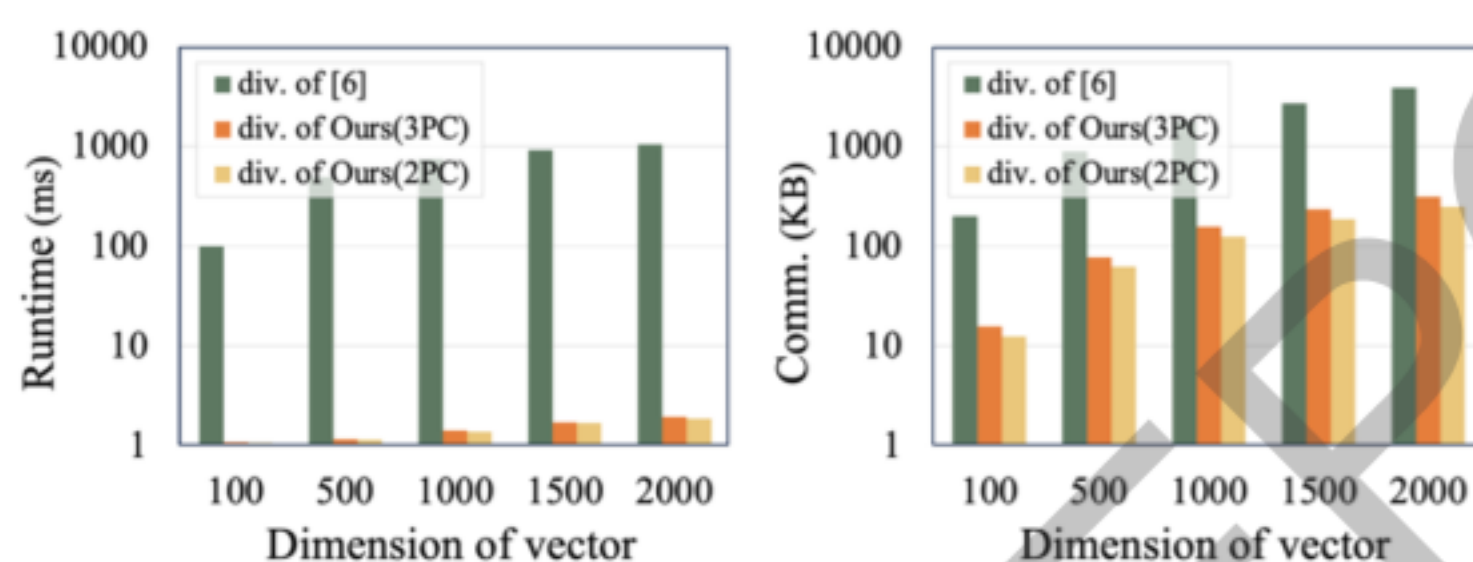


Figure 4 Comparing the runtime and communication overhead of division operation with PriVDT.

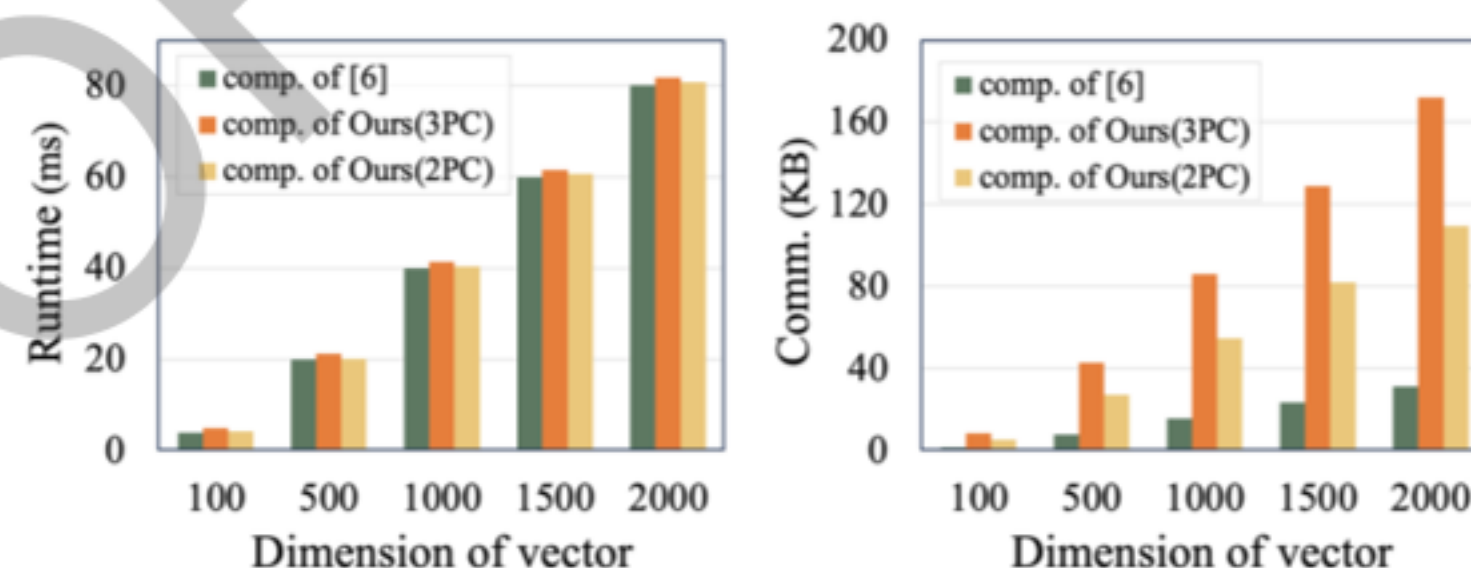
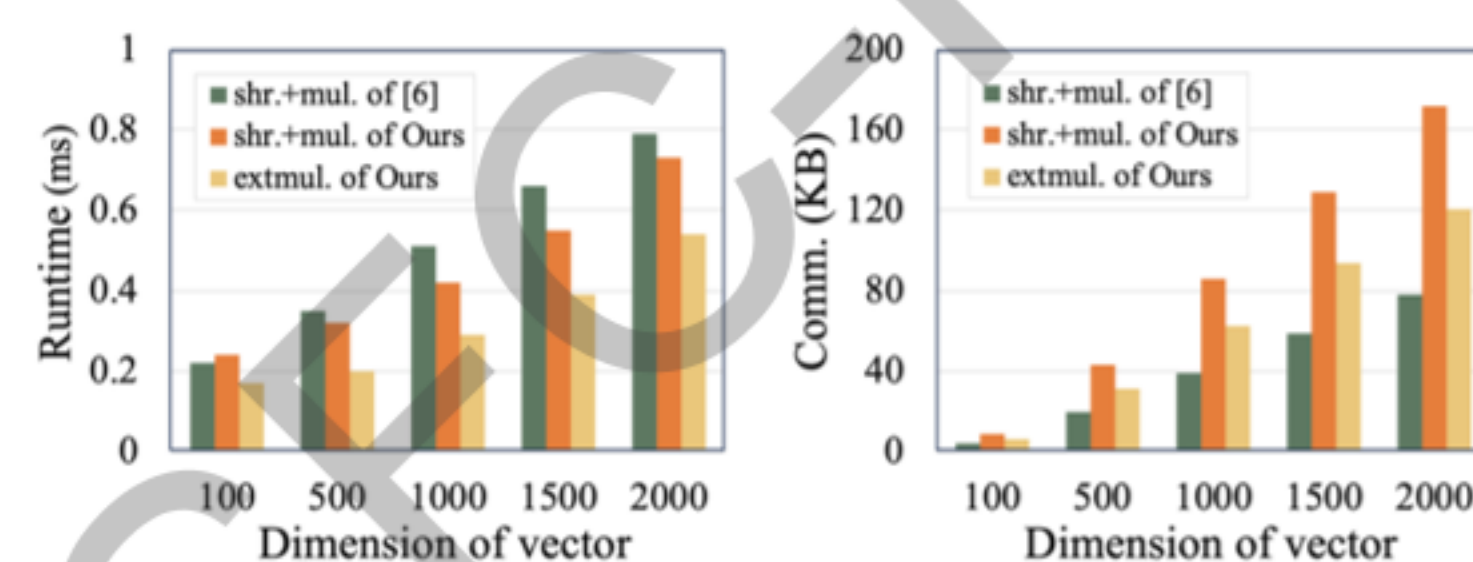
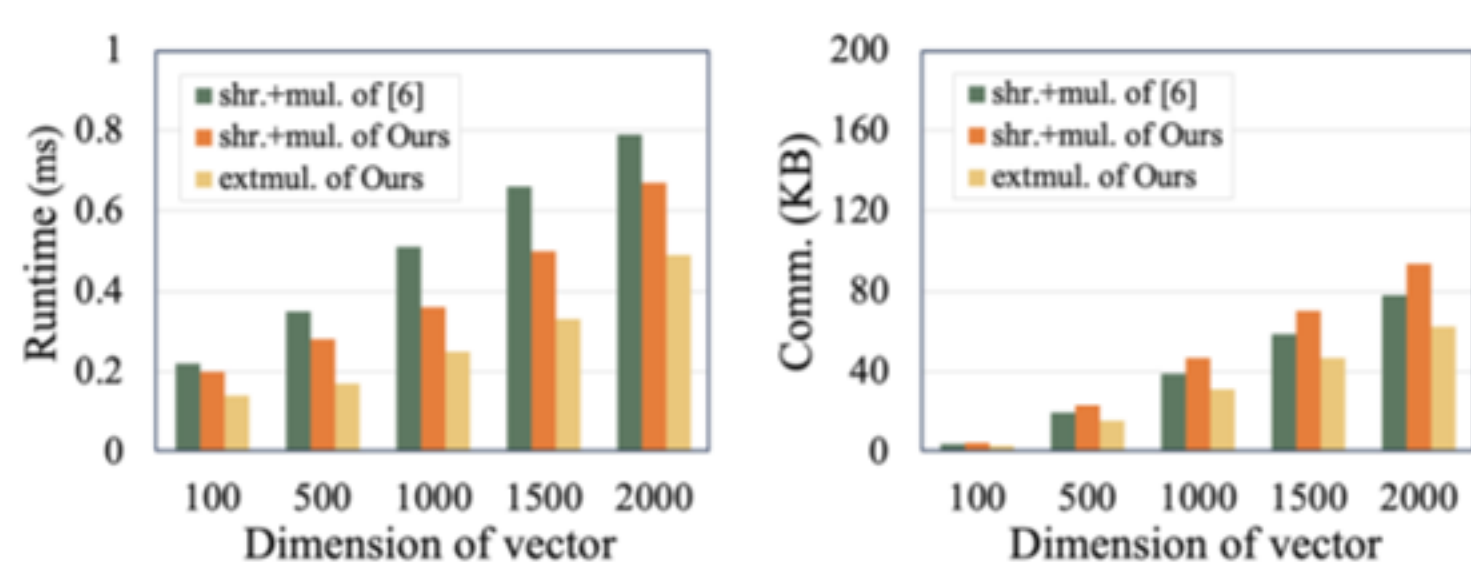


Figure 5 Comparing the runtime and communication overhead of comparison operation with PriVDT.



(a)



(b)

Figure 6 Comparing the runtime and communication overhead of multiplication operation with PriVDT. (a) Under the 3PC setting of SecureCART; (b) Under the 2PC setting of SecureCART.

研究成果以《Stateless Deterministic Multi-party EdDSA Signatures with Low Communication》、《Verifiable and Forward-Secure Multikeyword Query in Internet of Medical Things》、《Multi-party privacy-preserving decision tree training with a privileged party》、《Vertical Federated Convolutional Framework Based on Function Secret Sharing》等论文的形式正式发表在国际知名期刊/会议上。

### 2.2.2 多方神经网络隐私计算安全协议

项目组针对基于区域的卷积神经网络模型，设计安全除法协议（SDP）、安



全自然指数协议（SNEP）和安全自然对数协议（SNLP）等子协议和隐私保护安全 R-CNN 计算，确保在目标检测过程中各参与方的原始数据、模型参数以及中间结果不泄露；针对 LSTM 长短时记忆神经网络模型，结合“算术电路+布尔电路+函数秘密分享”混合框架，设计具有隐私保护的序列到序列转换；面向生成式预训练模型（GPT）模型，设计安全 Sigmoid、安全 GeLU 协议，以及针对大规模 Transformer 的隐私保护多方推理框架，探索多方神经网络隐私计算安全协议在大规模模型下的可行性和可推广性。

### （1）面向区域卷积神经网络的多方隐私保护计算协议

如图 1 所示，本项目设计的安全多方计算模型涉及六个主要实体：用户、模型提供者、三台服务器和一个可信第三方。在初始化阶段，模型提供者将模型参数（包括卷积层的权重等）秘密地分发给三台服务器。

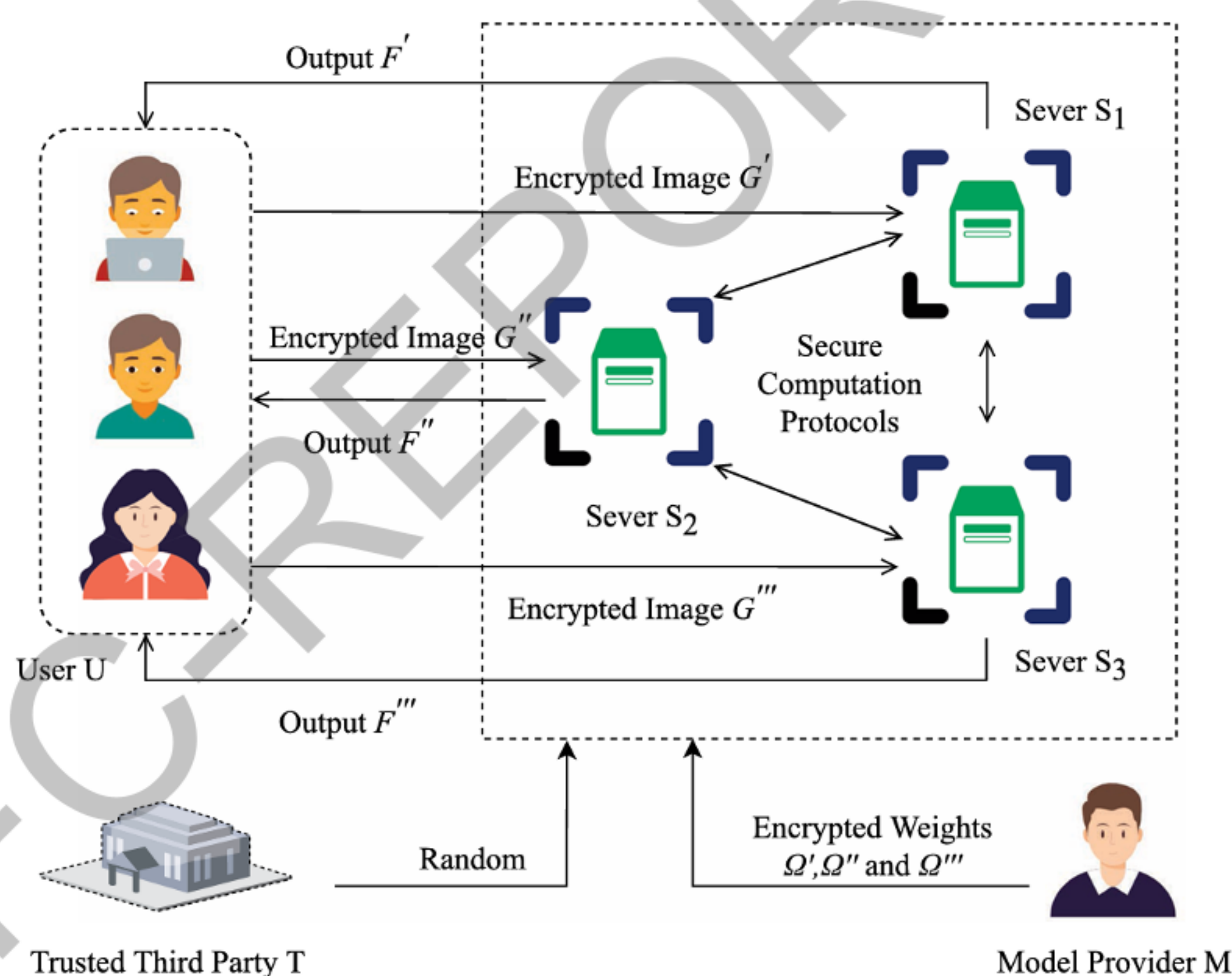


图 1 Faster RNN 系统模型

- 用户：用户  $U$  拥有原始图像  $G$ 。 $G$  被随机拆分为  $G = G' + G'' + G'''$ 。加密后的图像  $G', G''$  和  $G'''$  被分别发送到三台服务器  $S_1, S_2$  和  $S_3$  进行存储和计算。
- 模型提供者：模型提供者  $M$  拥有完整的权重  $\Omega$ 。 $\Omega$  被随机拆分为  $\Omega = \Omega' + \Omega'' + \Omega'''$ 。加密后的权重  $\Omega', \Omega''$  和  $\Omega'''$  被分别发送到三台服务器  $S_1, S_2$  和  $S_3$  进行存储和计算。。
- 服务器： $S_1, S_2$  和  $S_3$  是三台外包服务器，负责执行密集的计算任务。他



们基于图像和权重的份额，通过一系列关键的隐私保护协议完成 Faster R-CNN 的全部计算，而无需访问原始图像和权重。最终的输出  $F'$ ,  $F''$  和  $F'''$  通过安全通道发送给用户  $U$ 。然后，用户可以通过计算  $F = F' + F'' + F'''$  得到原始的预测结果。

可信第三方：可信第三方用于生成随机数。

为了实现原始图像的隐私保护以及三个服务器之间的安全交互，项目组设计了一系列基于盲化和 SMtA 协议的安全计算子协议，即安全除法协议（SDP）、安全自然指数协议（SNEP）和安全自然对数协议（SNLP）：

（一）安全除法协议（SDP）：

安全除法协议实现两个秘密共享值的除法操作，具体如下：

① 每个服务器  $S_i$  生成随机值  $[\rho]'$ ,  $[\rho]''$  和  $[\rho]'''$ ；

② 服务器  $S_1, S_2$  和  $S_3$  计算并广播值

$$(c', c'', c''') \leftarrow SMP([\rho]', [\rho]'', [\rho]''', [y]', [y]'', [y]''');$$

③ 服务器  $S_i$  计算  $c \leftarrow c' + c'' + c'''$ ；

④ 服务器  $S_1, S_2$  和  $S_3$  计算

$$([d]', [d]'', [d]''') \leftarrow SMP([\rho]', ([\rho]'', [\rho]''', [x]', [x]'', [x]'''));$$

⑤ 最后，服务器  $S_1, S_2$  和  $S_3$  计算并分别输出  $[z]' \leftarrow \frac{[d]'}{c}$ ,  $[z]'' \leftarrow \frac{[d]''}{c}$ , 和  $[z]''' \leftarrow \frac{[d]'''}{c}$ ；

（二）安全自然指数协议（SNEP）：实现自然指数函数的计算，即  $z = e^x$ ，其中  $x$  是一个秘密共享值，具体如下：

① 服务器  $S_1, S_2$  和  $S_3$  计算自然指数值：

$$\langle c \rangle' \leftarrow e^{[x]'},$$

$$\langle c \rangle'' \leftarrow e^{[x]''},$$

$$\langle c \rangle''' \leftarrow e^{[x]'''};$$

② 服务器  $S_1, S_2$  和  $S_3$  计算：

$$([z]', [z]'', [z]''') \leftarrow SMtA(\langle c \rangle', \langle c \rangle'', \langle c \rangle''');$$

（三）安全自然对数协议（SNLP）：实现自然对数函数的计算，即  $z = \log x$ ，



其中  $x$  是一个秘密共享值，具体如下：

离线阶段：

① 服务器  $S_1, S_2$  和  $S_3$  生成随机值  $\langle \rho \rangle', \langle \rho \rangle'', \langle \rho \rangle''' \in Z_{2^l}$ ；

② 服务器  $S_1, S_2$  和  $S_3$  计算：

$$([c]', [c]'', [c]''') \leftarrow \text{SMtA}(\langle \rho \rangle', \langle \rho \rangle'', \langle \rho \rangle''');$$

在线阶段：

③ 服务器  $S_1, S_2$  和  $S_3$  计算：

$$(k', k'', k''') \leftarrow \text{SMP}([c]', [c]'', [c]''', [x]', [x]'', [x]''');$$

④ 服务器  $S_2$  和  $S_3$  将  $k''$  和  $k'''$  发送给  $S_1$ ；

⑤ 服务器  $S_1$  计算：  $k \leftarrow k' + k'' + k'''$  和  $[z]' \leftarrow \log \frac{k}{\langle \rho \rangle'}$ ；

⑥ 服务器  $S_2$  和  $S_3$  分别计算：

$$[z]'' \leftarrow \log \frac{1}{\langle \rho \rangle''} \text{ 和 } [z]''' \leftarrow \log \frac{1}{\langle \rho \rangle'''};$$

项目组将介绍如何使用前述的安全计算子协议来构建一个隐私保护的 Faster R-CNN 框架。项目组的目标是确保在目标检测过程中，各参与方的原始数据、模型参数以及中间结果均不泄露。框架包含数据划分、安全特征提取、安全区域建议网络、安全分类。

（一）数据拆分：用户首先将尺寸为  $M \times N$  的图片  $G$  中的每个像素  $x_{m,n}$

随机拆分为  $x'_{m,n}$ 、 $x''_{m,n}$  和  $x'''_{m,n}$ ，使得  $x_{m,n} = x'_{m,n} + x''_{m,n} + x'''_{m,n}$ ，并获得图像的份额  $G'$ 、 $G''$  和  $G'''$ 。然后，将  $G'$ 、 $G''$  和  $G'''$  分别上传到服务器  $S_1$ 、 $S_2$  和  $S_3$ 。安全数据拆分（SDS）的具体步骤如下：

① 用户  $U$  输入图像  $G = \{x_{m,n,3}\}$  以及安全参数  $l$ ， $l$  定义了消息空间；

② 对于  $G$  中的每个像素，用户  $U$  生成两个随机值  $x'_{m,n,3}$  和  $x''_{m,n,3}$ ，并使其满足  $x'_{m,n,3}, x''_{m,n,3} \in [-2^{l-1}, 2^{l-1} - 1]$ ；

③ 用户  $U$  计算  $x'''_{m,n,3} = x_{m,n,3} - x'_{m,n,3} - x''_{m,n,3}$ ；

④ 图像  $G$  随机拆分为  $G'$ 、 $G''$  和  $G'''$ ，其中  $G' = \{x'_{m,n,3}\}$ ， $G'' = \{x''_{m,n,3}\}$  和  $G''' = \{x'''_{m,n,3}\}$ 。